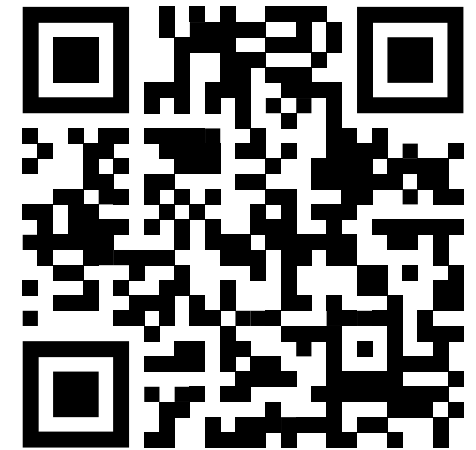


Programmierung in C++

Kopieren und Verschieben

Prof. Dr. Bernd Lüdemann-Ravit

Ansprechpartner: Vinzenz Funk



<https://poll.hs-kempten.de/poll/>



Wertausdrücke

Lernziele

- Die Aussage verschiedener **Ausdrücke** in Programmcode wurde hinsichtlich ihrer Werte verstanden
 - Objekte können nach ihrer **Adressierbarkeit** und **Veränderlichkeit** unterschieden werden
 - **Wertausdrücke** können aus Objektdeklarationen abgelesen werden

Wertausdrücke

Ausdrücke



- Ein **Ausdruck** ist in der Programmierung ein Konstrukt, das in Bezug auf einen Kontext ausgewertet werden kann und **einen Wert liefert**
- Häufige Ausdrücke sind:

Art des Ausdrucks	Beispiel
Literale (Konstanten)	2; 3.14; 42u; "Hallo"; 'F'
Variablen	x; texture; oger
Arithmetik	2*3; 2+3+5; 1/2
Zegerarithmetik	*(pString + 15); (&character) + 10
Funktionen	sin(3.14); random(); loadFromFile("wand.png")

- Ausdrücke können auch kombiniert betrachtet werden, z.B. der Ausdruck **sin(3.14)** in dem der Literalausdruck **3.14** und der Funktionsausdruck **sin()** steckt

Wertausdrücke

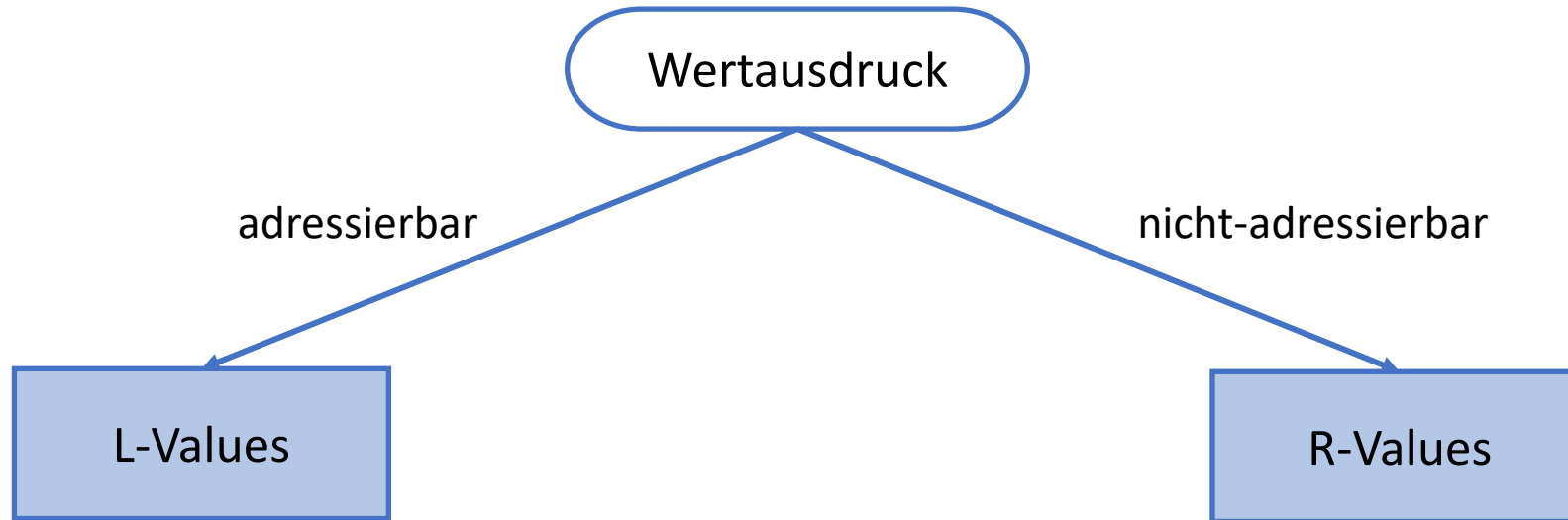


Kategorien (value categories)

- Bei Ausdrücken wird in C++ zwischen verschiedenen Arten der dargestellten Werte unterschieden (***value categories***)
 - Die Unterscheidung dient hauptsächlich der Kategorisierung, wie ein Ausdruck verwendet werden kann
- Entscheidend ist dabei, ob ein Ausdruck einen **adressierbaren Wert** darstellt – also sich an einer bestimmten Adresse im Prozessspeicher befindet
 - Adressierbare Ausdrücke verweisen i.d.R. auf einen **Speicherort** – meist eine Variable
 - Nicht-adressierbare Ausdrücke stellen dagegen häufig **Konstanten** oder **Funktions-Rückgabewerte** dar
- Die Arten der Werte (value categories) vermitteln dem **Compiler**, wie Ausdrücke zu **interpretieren** sind
 - Sie haben nichts direkt mit CPU-Operationen zu tun
 - Sie dienen mehr der **semantischen** Auswertung von Anweisungen

Wertausdrücke

L-Value und R-Value



Wertausdrücke



L-Value und R-Value

- Historisch bedingt (aus **C**), wird für die Bezeichnung der Wertausdrücke eine **Zuweisungsoperation** betrachtet:

```
int x = 42;
```

L-Value

R-Value

- Adressierbare Werte werden als **L-Value** bezeichnet, da diese *meist* links (**L**) von einer Zuweisungsoperation stehen
- Nicht-adressierbare Werte werden als **R-Value** bezeichnet, da diese *meist* rechts (**R**) von einer Zuweisungsoperation stehen

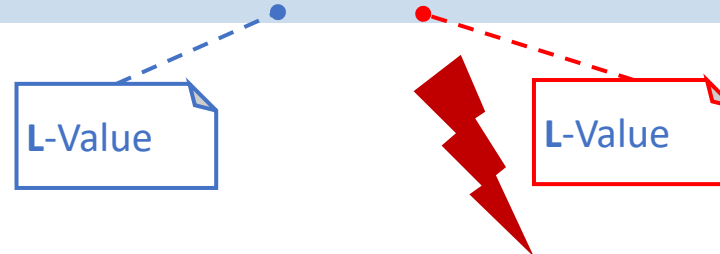
Wertausdrücke



L-Value und R-Value

- Die Betrachtung der Zuweisungsoperation ist jedoch missverständlich, da auch auf der rechten Seite ein adressierbarer Ausdruck stehen kann:

```
int y = 42;  
int x = y;
```



- Der Variablenausdruck **y** ist hier ebenfalls ein L-Value, da er einen adressierbaren Wert darstellt, obwohl er auf der rechten Seite der Zuweisung steht
- Nur das Literal **42** ist weiter ein R-Value

Wertausdrücke



L-Value und R-Value

- Die Kategorisierung von Werten wird bei allen Ausdrücken angewendet – beispielsweise auch bei einer Addition

$x + 2$ // OK: lvalue + rvalue
 $2 + x$ // OK: rvalue + lvalue



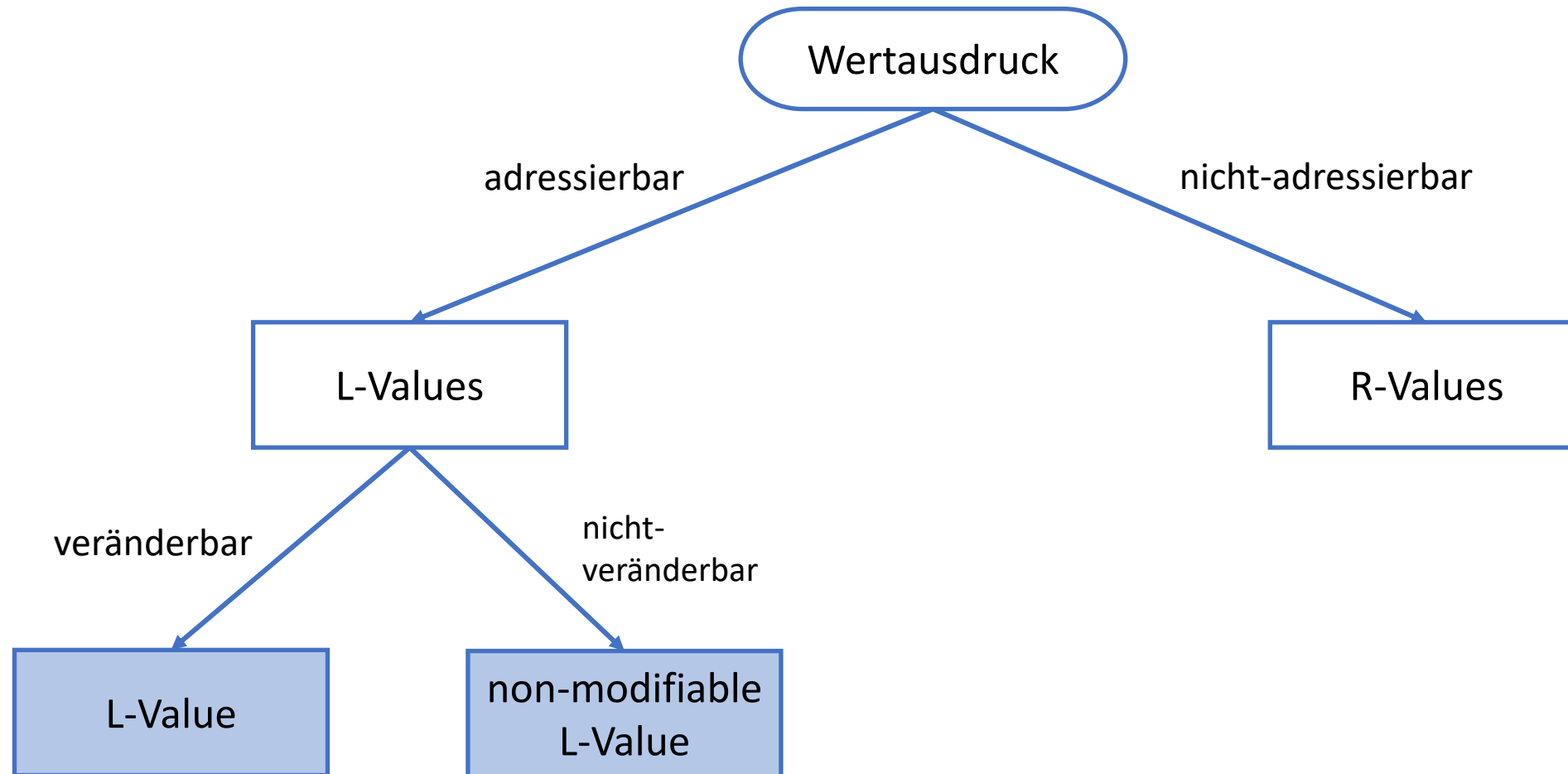
$x + 2 = 4$ // Fehler.. warum?



- Beide Operanden einer Addition können L-Values und R-Values sein
- Der gesamte Additionsausdruck ($x + 2$) stellt jedoch ein R-Value dar, da das Ergebnis der Addition **nicht adressierbar** ist
 - Es befindet sich wahrscheinlich in einem CPU-Register und ...
 - muss erst einem L-Value – beispielsweise einer lokalen Variable – zugewiesen werden, um adressiert werden zu können

Wertausdrücke

L-Value und R-Value – Veränderbarkeit





Wertausdrücke

L-Value und R-Value – Veränderbarkeit

- Adressierbarkeit bedeutet nicht Veränderbarkeit!

```
const char name[] = "vinzenz";  
name[0] = 'V'; // Fehler
```

- Der Wert des Ausdrucks **name** ist ein L-Value, da sich der String im Speicher befindet und einzelne Buchstaben daraus adressierbar sind
- Der String ist jedoch **const** deklariert und kann damit nicht verändert werden

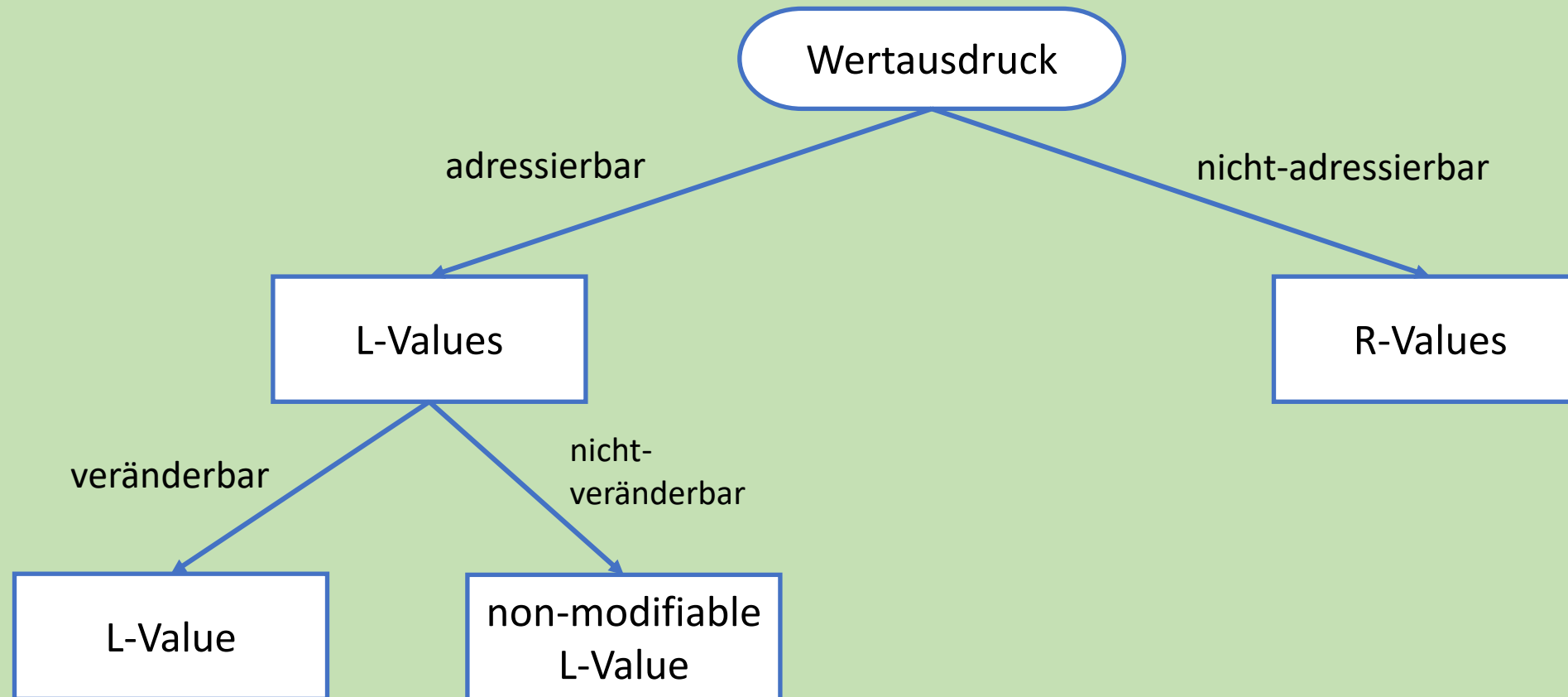
→ Diese L-Values heißen **non-modifiable L-Value**

Wertausdrücke

Zusammenfassung



- Übersicht Wertausdrücke (*bis jetzt*)





Referenzbindung

Lernziele

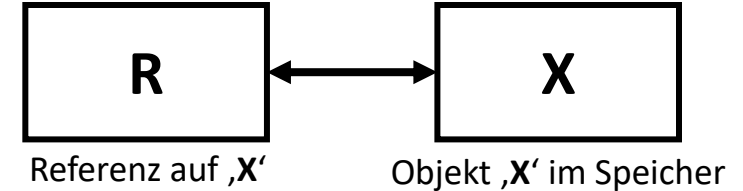
- Die Initialisierung von Referenzen mit Werten wurde verstanden
- Das Verhalten von **Referenzen auf Konstanten** wurde verstanden
 - Die Erstellung **temporärer Objekte** kann nachvollzogen werden
 - Mit temporären Objekten kann sicher umgegangen werden

Referenzbindung

Referenzen



- Rückblick: Eine **Referenz** ist ein Alternativname (*alias*) für ein Objekt
 - Referenzen können anstelle einer Variable selbst verwendet werden
 - Eine Referenz verhält sich wie ein Pointer, der bei Nutzung automatisch dereferenziert wird
 - ...



```
int x = 5;  
int& r = x; // Referenz der Variable 'x'  
std::cout << "x: " << x << ", r: " << r;  
x: 5, r: 5
```

```
r = 10; // Referenzierte Zuweisung  
std::cout << "x: " << x << ", r: " << r << std::endl;  
x: 10, r: 10
```

Referenzbindung



Bindung

- Wird ein Wert einer Referenzvariable zugewiesen, spricht man von „**Bindung** der Referenz“

```
int x = 42;  
int& r = x;
```

- Die **Bindung** unterscheidet sich von einer normalen Objektinitialisierung
 - Die Referenz selbst ist kein eigenes Objekt
 - Die Referenz selbst belegt keinen bekannten Speicherbereich
 - Der Compiler kann statt der Referenz das „Originalobjekt“ verwenden
 - Setzt der Compiler die Referenz als Zeiger um, kann er den Speicherort frei wählen
 - Bei der **Initialisierung** einer Referenz wird kein Objekt kopiert

Referenzbindung

Bindung



- Eine „normale“ Referenz kann nur auf L-Values gebunden werden
 - Da der Wert von non-modifiable L-Values und R-Values nicht verändert werden kann

```
int x = 42;  
✓ int& r1 = x; // Ok: L-Value  
const int y = 100;  
⚡ int& r2 = y; // Compilerfehler: non-modifiable L-Value  
⚡ int& r3 = 42; // Compilerfehler: R-Value
```

Referenzbindung



Referenzen auf Konstanten (reference to const)

- Referenziert eine Referenz eine Konstante, so können auch non-modifiable L-Values und R-Values gebunden werden

Referenz auf Konstante

```
int x = 42;
✓ const int& r1 = x; // Ok: L-Value
const int y = 100;
✓ const int& r2 = y; // Ok: non-modifiable L-Value
✓ const int& r3 = 42; // Ok: R-Value
```

... Moment ... Referenz auf ein R-Value?
R-Values sind doch nicht-adressierbar?

Referenzbindung



Temporäre Objekte

- Eine Referenz auf eine Konstante sagt aus, dass das referenzierte Objekt nicht verändert werden kann
 - Die Aussage ist nicht, dass das referenzierte Objekt adressierbar sein muss!
- Referenzen selbst müssen jedoch immer auch adressierbar sein, da sie L-Values sind
- Dies hat zur Folge, dass der Compiler für bestimmte Bindungen auf Referenzen temporäre Objekte erstellt!
 - Es handelt sich um „echte“ Objekte, die im Speicher liegen und adressiert werden können

Referenz auf Konstante

R-Value

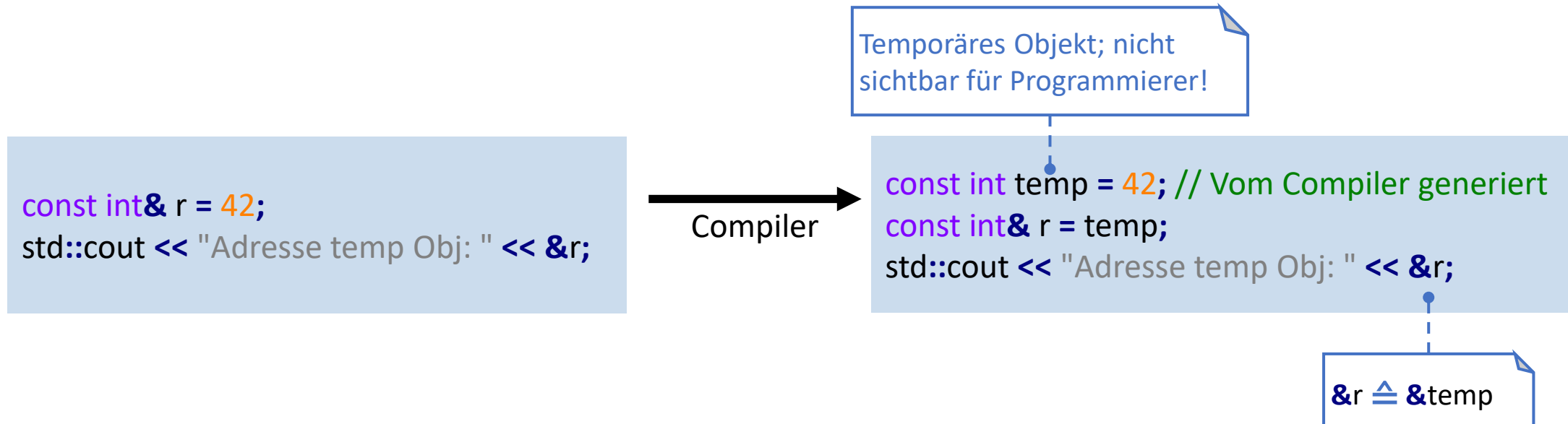
```
const int& r = 42; // Erstellt & referenziert temporäres Objekt
std::cout << "Adresse temp Obj: " << &r; // Ok: Adresse temp Obj: 00000022DFAFF6F4
std::cout << "Adresse R-Value: " << &42; // Compilerfehler: C2101 Der Adressoperator (&) muss
                                           ein L-Value als Operanden haben
```

Referenzbindung



Temporäre Objekte – Erzeugung

- Die Erzeugung temporärer Objekte geschieht **implizit**, ohne dass das Objekt händisch definiert werden muss
 - Die Referenz bindet dann dieses „versteckt“ erzeugte Objekt, nicht das R-Value!
 - R-Values bleiben nicht-adressierbar



Referenzbindung



Temporäre Objekte – Gültigkeit

- Temporäre Objekte sind nur innerhalb eines Ausdrucks gültig oder so lange, wie eine Referenz auf sie existiert
- Temporäre Objekte haben eine eigene Speicherdauer „**temporär**“; sie hängen von der Lebenszeit eines anderen Objekts (oder Ausdrucks) ab
 - Rückblick Speicherdauer:
 - **automatisch**: Das Objekt lebt so lange, wie es in seinem umschließenden Scope gültig ist
 - **dynamisch / manuell**: Das Objekt lebt so lange, bis es manuell durch den Programmierer gelöscht wird
 - **statisch**: Das Objekt lebt so lange, bis das gesamte Programm beendet wird
 - **temporär**: Das Objekt lebt so lange, bis es nicht mehr von Ausdrücken oder Referenzen verwendet wird

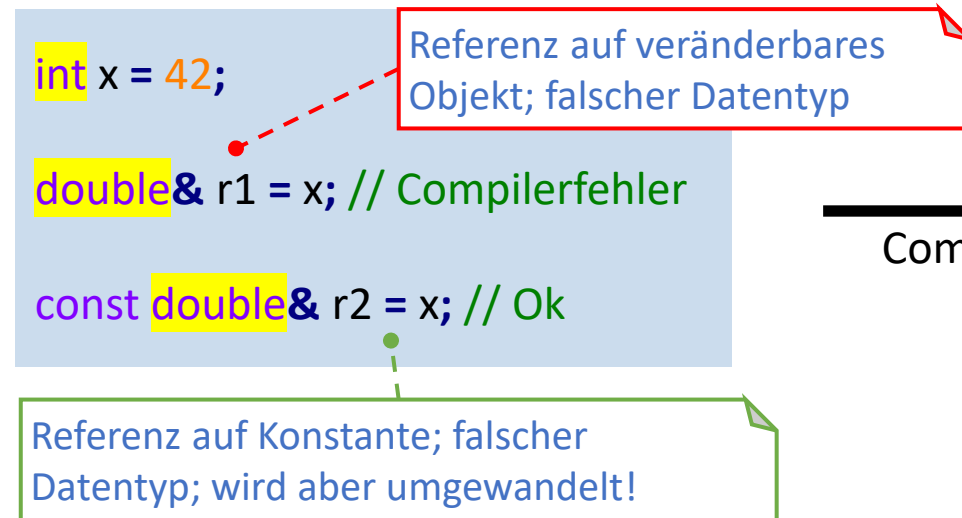
```
{  
    const int& r = 42;  
} // Scope-Ende zerstört Referenzvariable „r“ und darin-referenziertes temporäres Objekt
```

Referenzbindung



Typumwandlung durch Referenzen

- Referenzen auf veränderbare Objekte können nur Objekte desselben Datentyps binden
- Referenzen auf Konstanten können auch Objekte binden, die zum Typ des Referenzziels **umgewandelt** werden können
 - Dabei wird ein temporäres Objekt für das Resultat der Umwandlung erstellt – wie bei der Zuweisung von R-Values



„Versteckte“ Umwandlung `int` → `double`

```
int x = 42;  
double temp = static_cast<double>(x);  
const double& r2 = temp;
```

Referenzbindung



Typumwandlung durch Referenzen – Beispiel

- Implizite Typumwandlung durch Referenz bei der Objektübergabe an eine Funktion

Referenz auf veränderliches Objekt

```
void print(std::string& s)
{
    std::cout << s;
}

int main()
{
    char text[] = "bla";
    print(text); // Compilerfehler
}
```

Referenz auf Konstante

```
void print(const std::string& s)
{
    std::cout << s;
}

int main()
{
    char text[] = "bla";
    print(text); // Ok
}
```

Implizite Umwandlung
`char* → std::string`
über temporäres Objekt

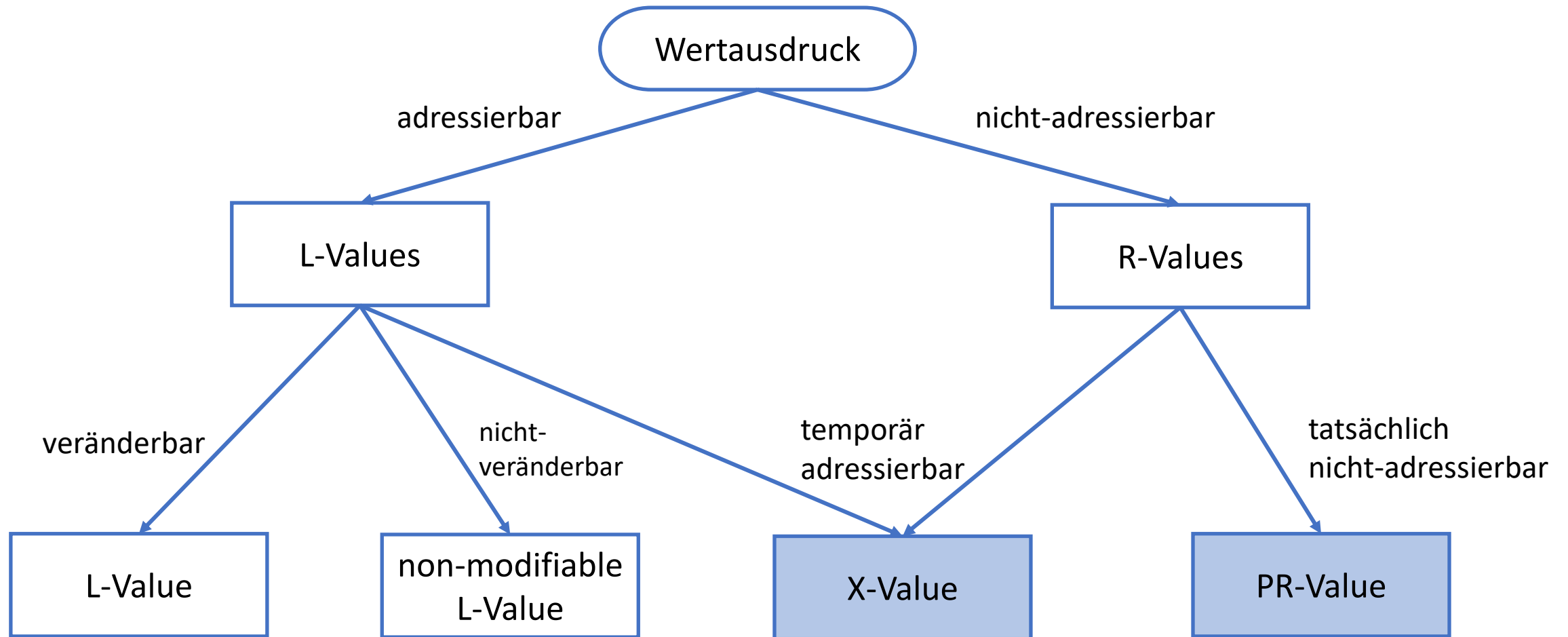


Enthält versteckten Konstruktoraufruf & Allokation!



Referenzbindung

X-Values & PR-Values



Referenzbindung



X-Values & PR-Values

- Ein temporäres Objekt, das in einer Referenz gebunden ist, heißt „**X-Value**“
 - Steht für „**eXpiring Value**“ (= schnell verfallender Wert)
- X-Values stellen einen **eigenen Wertausdruck** dar, der Eigenschaften von L-Values und von R-Values besitzt
 - **Ähnlichkeit zu L-Values**: X-Values sind „echte“ Objekte an einem Speicherort, deren Adresse mit dem Adressoperator (&) erhalten werden kann
 - **Ähnlichkeit zur R-Values**: X-Values existieren nur temporär für eine kurze Dauer und haben keinen Namen
- R-Values, die tatsächlich nicht-adressierbar sind, werden „**PR-Values**“ genannt
 - Steht für „**Pure R-Value**“

```
const int& r = 42;
```

„r“ enthält X-Value

Literal „42“ ist ein PR-Value

Referenzbindung

X-Values & PR-Values



- Temporäre Objekte werden auch ohne Referenzbindung vom Compiler erstellt
 - Sie leben dann nur für die Dauer eines einzigen Ausdrucks! (= PR-Value)
- Referenzen verlängern die Lebensdauer temporärer Objekte (= X-Value)

```
int main()
{
    42; // PR-Value; temporäres Objekt
    // Objekt hier bereits wieder zerstört
}
```

Für den Literal-Ausdruck „42“ wird ein `int`-Objekt erzeugt und in der selben Zeile wieder zerstört = **temporäre Speicherdauer bei PR-Values**

```
int main()
{
    const int& r = 42; // PR-Value → X-Value

    ... // anderer Programmcode...

} // Objekt wird erst hier zerstört
```

Die Referenz bindet das temporäre `int`-Objekt und hält es am Leben, bis die Gültigkeit der Referenz selbst endet – also bis zum Ende des umschließenden Scopes = **temporäre Speicherdauer bei X-Values**

Referenzbindung

Zusammenfassung



- Bei der **Referenzbindung** wird eine Referenzvariable mit einem Wert initialisiert
 - Anders als bei der Objektinitialisierung entsteht dadurch nicht zwingend ein neues Objekt
- **Referenzen auf Konstanten** erlauben auch die Referenzierung von R-Values und des Resultats von Typumwandlungen
 - Hierbei entstehen temporäre Objekte, die nur so lange wie der Referenzausdruck selbst existieren (**X-Values**)
- **Temporäre Objekte** werden nach ihrer Adressierbarkeit unterschieden
 - **X-Value**: adressierbar, so lange von einer Referenz verwiesen
 - **PR-Value**: nicht-adressierbar, anonym



Referenzausdrücke

Lernziele

- Die Aussage von **Referenzausdrücken** auf verschiedene Arten von Werten wurde verstanden
- Insbesondere wurde ein Verständnis für die neuartigen **R-Value-References** geschaffen, um moderne Sprachkonzepte nachvollziehen zu können



Referenzausdrücke

Referenz-Zeiger-Dualität

- Referenzen verhalten sich wie konstanter Zeiger:

Als Referenz	Als Pointer
<code>int& rx = x;</code>	<code>int *const px = &x;</code>
<code>rx = 42;</code>	<code>*px = 42;</code>
<code>int y = rx + 2;</code>	<code>int y = *px + 2;</code>

Der Zeiger ist konstant, nicht der Datentyp int!

- Die Adresse eines referenzierten Objekts kann mit dem Adressoperator (&) erhalten werden

Als Referenz	Als Pointer
<code>int* p = &rx;</code>	<code>int* p = &(*px);</code>

$\&(*px) \triangleq px$



Liefert nicht die Adresse der Referenz selbst!



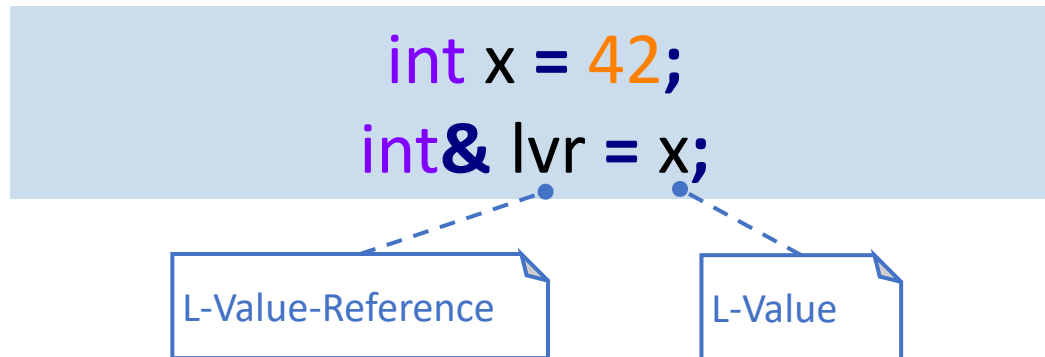
- Daher werden Referenzen selbst als **L-Value** Ausdrücke betrachtet

Referenzausdrücke



Referenzen – Wertausdruck

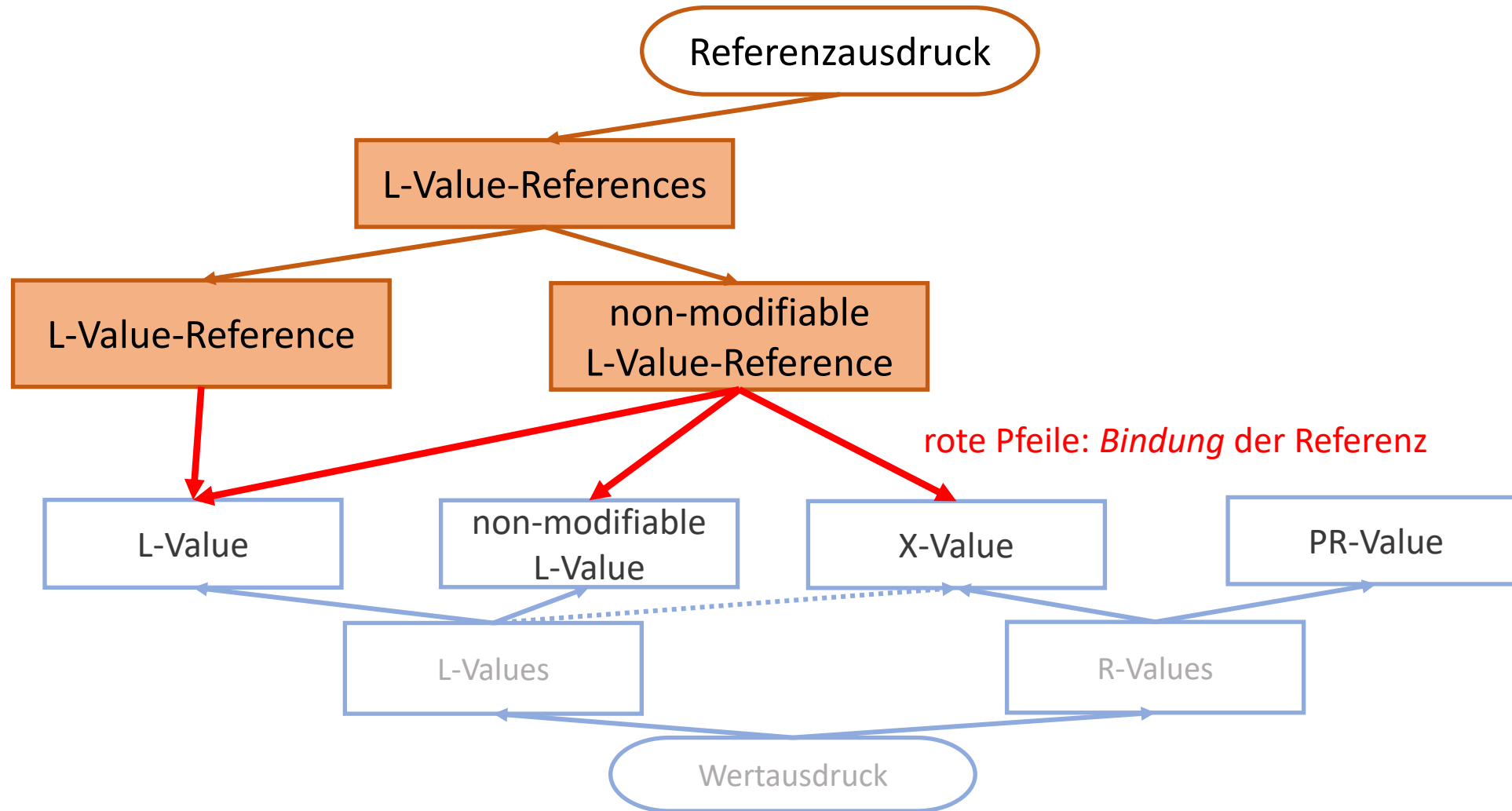
- Referenzen werden hinsichtlich der Werte, die sie referenzieren, betrachtet
 - Nicht des Werts der Referenz selbst! (wäre immer ein L-Value)
 - Betrachtet wird der Wert des Variablenausdrucks, anstelle dessen die Referenz verwendet werden kann



- Der Wert **`lvr`** kann wie das L-Value **`x`** verwendet werden (= Alias) und ist damit eine **L-Value-Reference**

Referenzausdrücke

L-Value-References



Referenzausdrücke



L-Value-References

- Eine **L-Value-Reference** referenziert ein veränderliches Objekt (L-Value)
- Eine **non-modifiable L-Value-Reference** (auch „*reference to const*“ genannt) referenziert...
 - Veränderliche Objekte (L-Values),
 - Konstante Objekte (non-modifiable L-Values),
 - Temporäre Objekte (X-Values)

L-Value-Reference

```
int lv = 42;  
const int nmlv = 42;
```

```
✓ int& lvr1 = x; // Ok: L-Value  
✗ int& lvr2 = nmlv; // Compilerfehler: non-modifiable L-Value  
✗ int& lvr3 = 42; // Compilerfehler: PR-Value
```

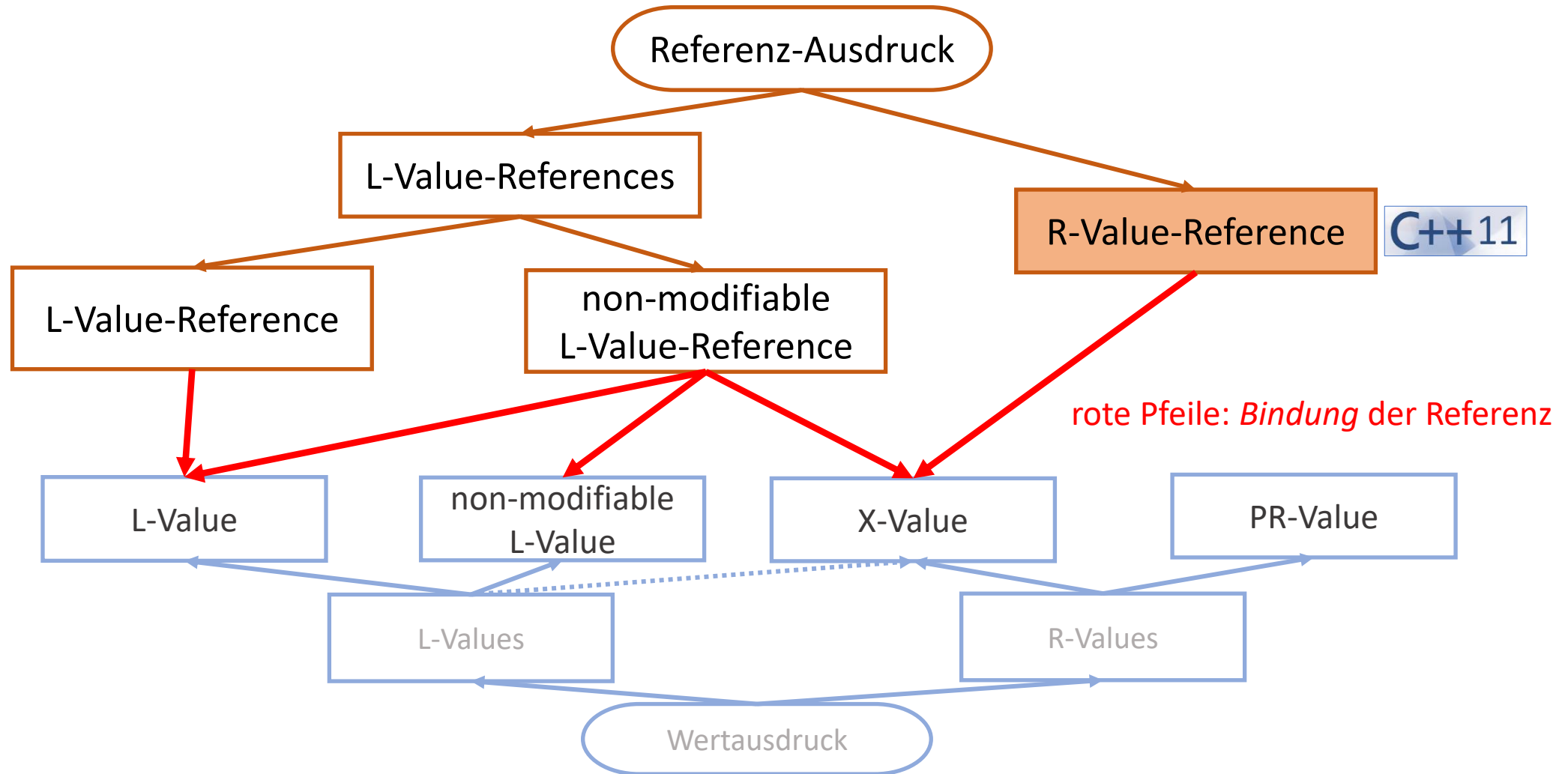
non-modifiable L-Value-Reference

```
int lv = 42;  
const int nmlv = 42;
```

```
✓ const int& nmlvr1 = x; // Ok: L-Value  
✓ const int& nmlvr2 = nmlv; // Ok: non-modifiable L-Value  
✓ const int& nmlvr3 = 42; // Ok: PR-Value → X-Value
```

Referenzausdrücke

R-Value-Reference



Referenzausdrücke



R-Value-Reference

- Bis zum **C++11**-Standard gab es keine Unterscheidung von Referenzausdrücken
 - Alle Referenzen waren L-Value-References (veränderbar und nicht-veränderbar)
 - R-Values wurden immer über X-Values in Referenzen gebunden
- Um die Erweiterung der Sprache um moderne Konstrukte zu ermöglichen, führt der **C++11**-Standard einen neuen Referenztyp ein, die **R-Value-Reference**
- R-Value-References ermöglichen die Semantik von **Verschiebungsoperationen** in modernem C++ (*Erklärung folgt*)

Referenzausdrücke



R-Value-Reference

- **C++11** Eine Referenz auf ein R-Value, wird als **R-Value-Reference** bezeichnet
- Syntaktisch werden R-Value-References durch ein **doppeltes Referenzsymbol (&&)** dargestellt

```
int&& rvr = 42;
```

R-Value-Reference

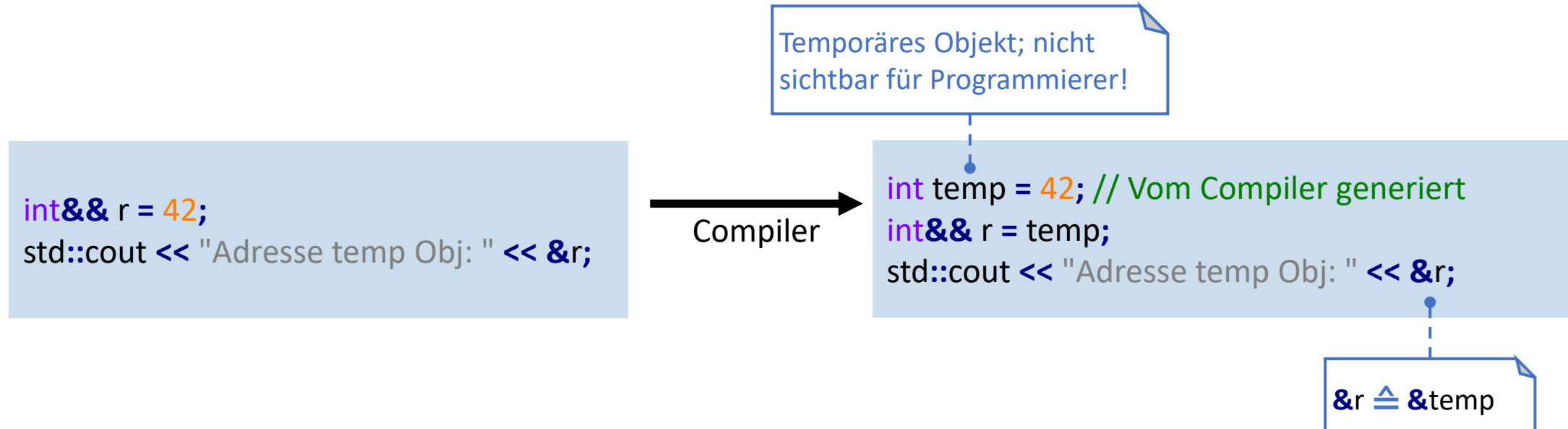
R-Value



Referenzausdrücke

R-Value-Reference – Bindung

- R-Value-References binden R-Values über **temporäre Objekte**
 - Wie bei der Bindung von non-modifiable L-Value-References auf R-Values
 - Diese temporären Objekte werden auch „**Verkörperungen**“ (*materializations*) der R-Values genannt



... Also genau wie Referenzen auf Konstanten?

Referenzausdrücke



R-Value-Reference vs. non-modifiable L-Value-Reference

- R-Value-References binden sich ausschließlich auf R-Values

R-Value-Reference

```
int lv = 42;  
const int nmlv = 42;
```

```
⚡ int&& rvRef1 = x; // Compilerfehler: L-Value  
⚡ int&& rvRef2 = nmlv; // Compilerfehler: non-modifiable L-Value  
✓ int&& rvRef3 = 42; // Ok: PR-Value → X-Value
```

non-modifiable L-Value-Reference

```
int lv = 42;  
const int nmlv = 42;
```

```
✓ const int& nmlvr = x; // Ok: L-Value  
✓ const int& nmlvr = nmlv; // Ok: non-modifiable L-Value  
✓ const int& nmlvr = 42; // Ok: PR-Value → X-Value
```

⚠ Durch Typumwandlungen entstandene temporäre Objekte werden auch von R-Value-References gebunden ⚠

```
double wrongType = 3.14;  
int&& rvRef4 = wrongType; // Ok: L-Value → Typumwandlung → X-Value
```

Referenzausdrücke



R-Value-Reference vs. non-modifiable L-Value-Reference

- Werte, die in R-Value-References gehalten werden, sind veränderlich

R-Value-Reference

```
int&& rvRef3 = 42; // Referenziert keine Konstante!  
✓ rvRef3 = 5; // Ok: Wert kann verändert werden
```

non-modifiable L-Value-Reference

```
const int& rvRef3 = 42;  
⚡ rvRef3 = 5; // Compilerfehler, da Konstante  
referenziert wird
```

Referenzausdrücke



R-Value-Reference – Nutzen

- Was ist der Mehrwert von R-Value-References gegenüber non-modifiable L-Value-References (= reference to const) ?

R-Value-References garantieren, dass es sich bei dem referenzierten Wert um ein **temporäres Objekt** handelt

- Im Gegensatz zu non-modifiable L-Value-References, lassen sich R-Value-References **nicht auf ein tatsächliches L-Value** bzw. Objekt binden!
 - Die gebundenen Objekte sind immer vergänglicher Natur
- Funktionen können, durch R-Value-References referenzierte Objekte beliebig bearbeiten, ohne dass dabei ein tatsächliches Objekt verändert bzw. beschädigt wird

Referenzausdrücke



R-Value-Reference – Beispiel

- Beispiel: Veränderung an einer R-Value-Reference (= dem temporären Objekt, auf das verwiesen wird)

```
int main()
{
    double orig = 3.14;

    int&& rvRef4 = orig; // temporäres Objekt
    rvRef4 = 42; // Änderung an temporärem Objekt

    std::cout << "Orig: " << orig
              << ", rvRef: " << rvRef4;
}
```

Das X-Value wird referenziert,
nicht das L-Value `orig`

Orig: 3.14, rvRef: 42

→ Originalwert wurde durch
Änderung über R-Value-
Reference nicht verändert

Referenzausdrücke



R-Value-Reference – Nutzen

- Der eigentliche Nutzen von R-Value-References ergibt sich bei **Verschiebungsoperationen** von komplexen Objekten
 - Da Objektreferenzen als „verwerfbar“ deklariert werden können
- Bei elementaren Datentypen (int, float, double, ...) sind Kopien schnell und bräuchten die zusätzlich eingeführte Komplexität nicht

Referenzausdrücke



Priorität der Bindung

- L-Value und X-Value Wertausdrücke können durch mehrere Referenzausdrücke gebunden werden (*siehe Graph*)
 - **(modifiable) L-Values:** von L-Value-References und non-modifiable L-Value-References
 - **X-Values:** von non-modifiable L-Value-References und R-Value-References
- Stehen mehrere Referenzausdrücke für die Bindung zur Auswahl, werden Objektausdrücke **zuerst** durch den Referenzausdruck gebunden, der der Deklaration des Objekts am ähnlichsten ist

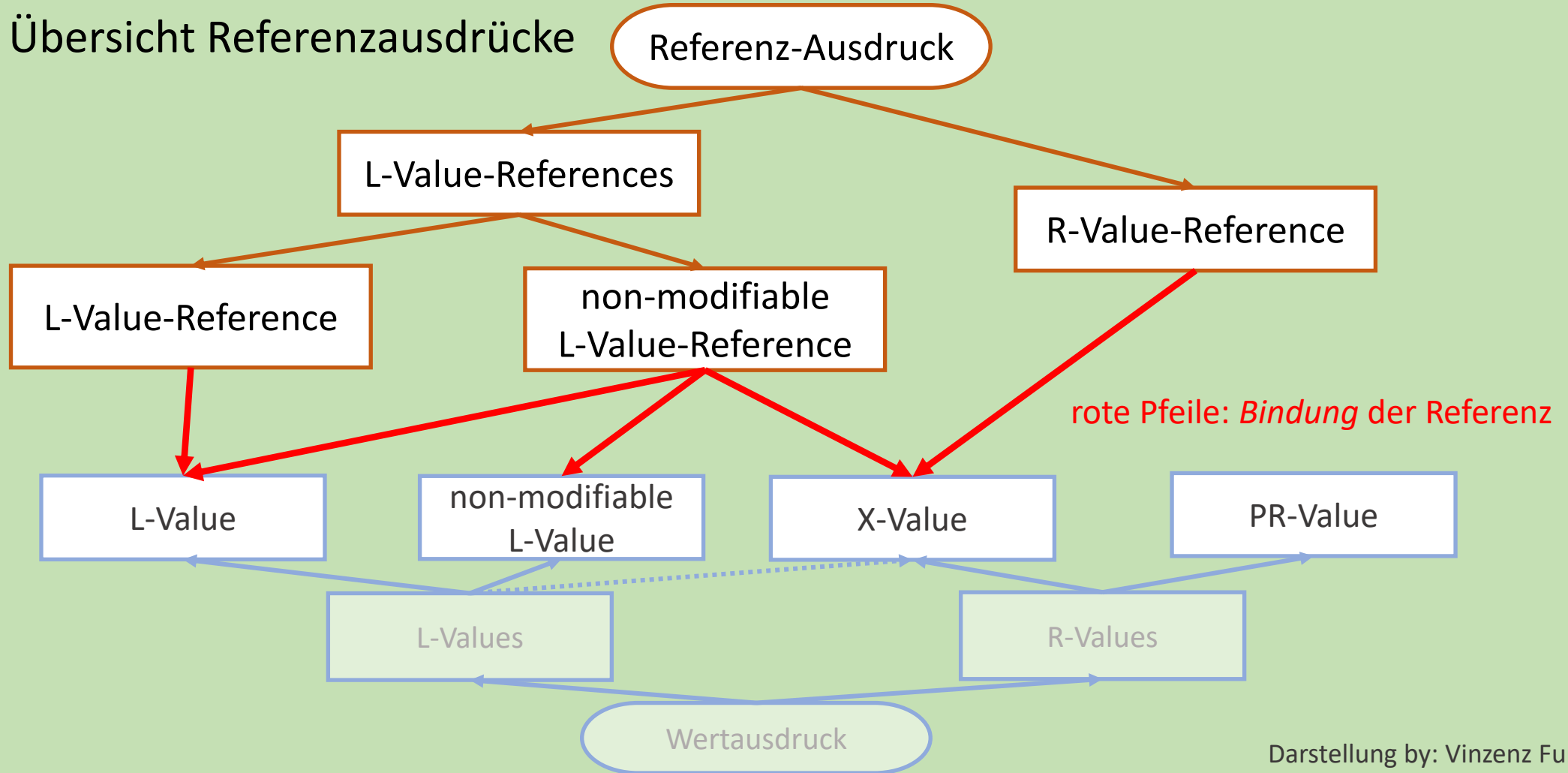
	Bevorzugter Referenzausdruck	Beispiel
(modifiable) L-Value	(modifiable) L-Value-Reference	<code>int lv = 42;</code> <code>int& lvr = x;</code>
Non-modifiable L-Value	Non-modifiable L-Value-Reference (<i>einzige Option</i>)	<code>const int nmlv = 42;</code> <code>const int& nmlvr = x;</code>
X-Value	R-Value-Reference	<code>int&& rvr = 42;</code>

Referenzausdrücke

Zusammenfassung



■ Übersicht Referenzausdrücke



Darstellung by: Vinzenz Funk

Wertausdrücke & Referenzen

Verständnisfragen

PIN: 327203

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Ein Ausdruck bezieht sich immer auf eine Variable.
- Auf der linken Seite einer Zuweisung steht immer ein L-Value.
- Auf der rechten Seite einer Zuweisung steht immer ein R-Value.
- Eine Referenz ist ein eigenes Objekt, das im Speicher liegt.
- Referenzen können mit PR-Values initialisiert werden.
- Referenzen können PR-Values binden.
- Objekte, die von R-Value-References referenziert werden, sind immer vergänglicher Natur.

■ Welche Eigenschaften haben X-Values?

- Sie existieren an einem konkreten Speicherort.
- Sie sind adressierbar.
- Sie sind veränderbar.
- Sie existieren nur für eine sehr kurze Zeit.



Kopiersemantik

Lernziele

- Objektinstanzen (Klassen) werden dahingehend betrachtet, wie sie sich bei einer **Zuweisung** verhalten
 - Es kann nachvollzogen werden, wann eine Objektinstanz **kopiert** wird und wie dieses Verhalten **gesteuert** werden kann
 - Im Zuge dessen wird nachvollzogen, warum und wie im Detail **Kopiersemantiken** funktionieren
- Explizite Kopieroperationen können für eigene Klassen definiert und optimiert werden

Kopiersemantik



Nutzen

- Je nachdem, wie Wert- und Referenzausdrücke einander **zugewiesen** werden, entstehen **verschiedene Kopiermuster** von Objekten
- Traditionell gab es nur die Möglichkeit Objekte bei der Zuweisung bzw. Erstellung zu **kopieren** (*copy*)
- Seit **C++11** ist es ebenfalls möglich, Objekte zu **verschieben** (*move*)
- Eines der Ziele von C++ ist, dass komplexe Objekte wie elementare Datentypen genutzt werden können
 - Dazu wurden **Semantiken** zum **Zuweisungsverhalten** von Objekten eingeführt

```
int main()
{
    int n1 = 42;
    int n2 = n1;
}
```

elementarer Datentyp: `int`

✓ alles klar

```
int main()
{
    std::string s1 = "Hallo";
    std::string s2 = s1;
}
```

komplexes Objekt: `string`

⚠ kopiere alle Zeichen?

⚠ kopiere Pointer auf Zeichen?

⚠ ... ?

Kopiersemantik

Objekterstellung



- Ein Objekt wird **erstellt** (***constructed***), wenn der Konstruktor eines Objekts aufgerufen wird
- Das wird hervorgerufen durch...
 - Die reine Deklaration eines Objekts (Standardkonstruktor)
 - Die Angabe von Konstruktorargumenten in runden oder eckigen Klammern
 - Die Zuweisung zur Zeit der Deklaration – in einer Anweisung

```
int main()
{
    Number n1; // Erstellung / Konstruktoraufruf
    Number n2 { 42 }; // Erstellung / Konstruktoraufruf
    Number n3 = 42; // Erstellung / Konstruktoraufruf
}
```

Trotz Zuweisungsoperator wird hier der **Konstruktor** aufgerufen bzw. das Objekt in einem Schritt **erstellt**!

```
class Number
{
public:
    Number(int n) : m_n{ n } {}
    Number() : m_n{ 0 } {}

private:
    int m_n;
};
```

Kopiersemantik

Kopierkonstruktor



- Der **Kopierkonstruktor** erstellt die Instanz der Klasse als Kopie aus einer anderen Instanz der Klasse
 - Dazu erhält er ein Argument: Die **non-modifiable L-Value-Reference** auf die Instanz, aus der kopiert werden soll
 - Der Kopierkonstruktor darf also das referenzierte Objekt nicht verändern!
- Der Compiler generiert automatisch einen Kopierkonstruktor für eine Klasse, der jede einzelne Membervariable der Klasse kopiert

```
class Number
{
public:
    Number(int n) : m_n{ n } {}
    Number() : m_n{ 0 } {}
    Number(const Number& other)
    {
        m_n = other.m_n;
    }

private:
    int m_n;
};
```

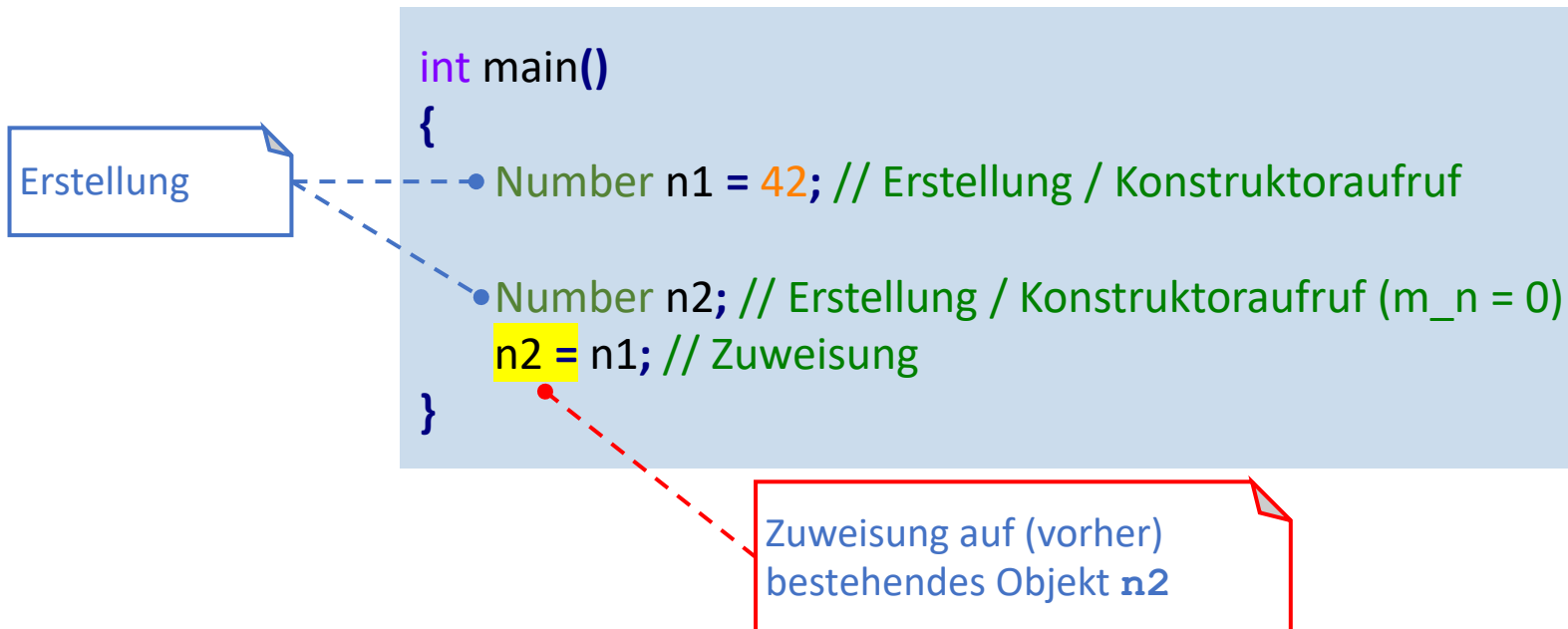
Kopierkonstruktor

Kopiersemantik



Zuweisung

- Ein Objekt wird **zugewiesen** (*assigned*), wenn ein bereits **bestehendes Objekt** auf der **linken Seite** der Zuweisungsoperation steht



Kopiersemantik

Kopier-Zuweisungsoperator



- Der **Kopier-Zuweisungsoperator** kopiert auf die eigene vorhandene Instanz (**this**) die Daten aus einer anderen Instanz, der gleichen Klasse
 - Dazu erhält er – wie der Kopierkonstruktor – eine **non-modifiable L-Value-Reference** auf die Instanz, die zugewiesen & kopiert werden soll
- Der Compiler generiert automatisch einen Kopier-Zuweisungsoperator für eine Klasse, der jede Membervariable kopiert – wie beim Kopierkonstruktor

```
class Number
{
public:
    Number(int n) : m_n{ n } {}
    Number() : m_n{ 0 } {}
    Number(const Number& other)
    {
        m_n = other.m_n;
    }
    Number& operator=(const Number& other)
    {
        m_n = other.m_n;
        return *this;
    }

private:
    int m_n;
};
```

Kopier-
Zuweisungsoperator

Kopiersemantik



Zuweisungsoperatoren

- Zuweisungsoperatoren (*assignment operators*) geben eine **veränderbare Referenz auf das eigene Objekt** zurück

```
Number& operator=(const Number& other)
{
    m_n = other.m_n;
    return *this;
}
```

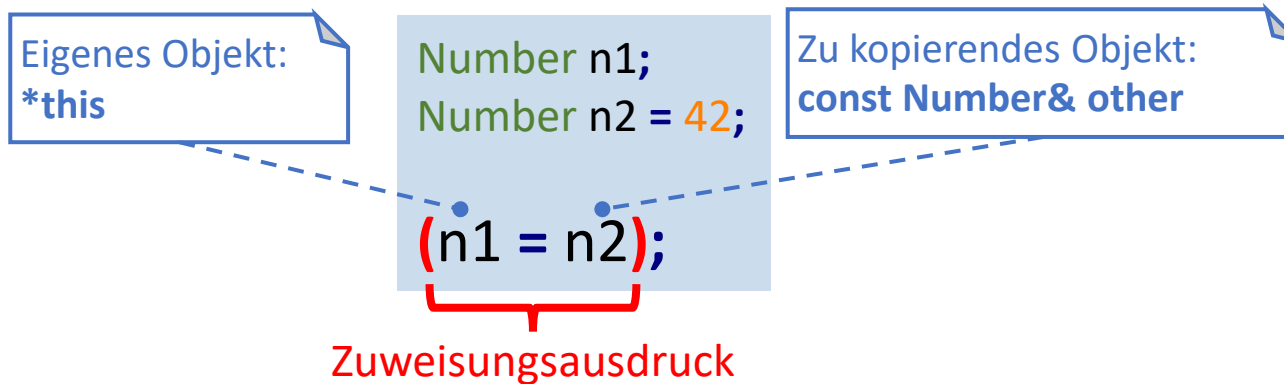
Warum?

Kopiersemantik



Zuweisungsoperatoren

- Das „Ergebnis“ einer Zuweisungsoperation ist der Wert des Zuweisungsausdrucks



- In den meisten Fällen könnte der Rückgabewert von Zuweisungsoperatoren „**void**“ sein, da mit dem Resultat einer Zuweisung nicht weiter gearbeitet wird
 - Werden mehrere Zuweisungen jedoch **verkettet**, ist es von Vorteil, wenn die Rückgabewerte **L-Values-References** darstellen, auf die **zugewiesen** werden kann

```
Number n1 = 5, n2 = 7, n3 = 42;  
(n1 = n2) = n3;
```

n1: 42
n2: 7
n3: 42

Kopiersemantik



Zuweisungsoperatoren – Verkettung

■ Beispiele: Verkettete Zuweisungen

```
Number n1 = 5;  
Number n2 = 7;  
Number n3 = 42;  
  
(n1 = n2) = n3;
```

Zuweisung auf das
Ergebnis von (n1 + n2)

n1: 42
n2: 7
n3: 42

- Wäre der Rückgabewert der Zuweisung...
 - „**void**“, so würde ein Compilerfehler auftreten,
 - ein **L-Value ohne Referenz**, so würde die Operation **(n1 = n2)** ein temporäres Objekt erstellen, das sofort nach der Zuweisung zerstört wird
→ **n3 auf n1** wird nicht zugewiesen
- Außerdem würde jede Rückgabe ohne Referenz eine **unnötige Kopie** auf das eigene Objekt erstellen

```
n1 = n2 = n3;  
n1 = (n2 = n3);
```

Resultat:
unnötige Kopie

n1: 42
n2: 42
n3: 42

Kopiersemantik



Kopien

- Bei manchen Objekten ist das implizite Kopierverhalten (Konstruktion und Zuweisung) problematisch
 - Bei implizit, durch den Compiler generierten, Kopierkonstruktoren und Kopier-Zuweisungsoperatoren, wird jede Membervariable **trivial kopiert, auch Zeiger**
→ Dabei spricht man auch von einer „**shallow copy**“ (oberflächliche Kopie)
 - Das ist besonders problematisch bei Objekten, die **Ressourcen mit dynamischer Speicherdauer verwalten** (z.B. Zeiger auf dynamisch erzeugte(s) Objekt(e) mit `new/delete`)
- Daher ist es möglich, einen Kopierkonstruktor und Kopier-Zuweisungsoperator **explizit zu definieren**

Kopiersemantik

Kopien – Beispiel

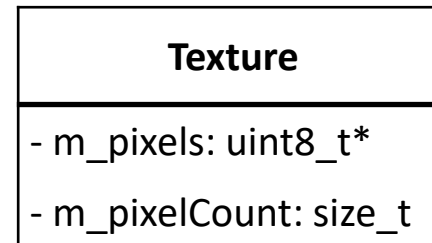


- Beispiel: Einem Texture-Objekt liegen Pixeldaten mit dynamischer Speicherdauer zugrunde, die über einen Zeiger verwaltet werden

```
class Texture
{
public:
    void loadFromFile(std::string file)
    {
        m_pixelCount = getPixelCount(file);
        m_pixels = new uint8_t[m_pixelCount];
        ...
    }
    ~Texture()
    {
        delete[] m_pixels;
    }

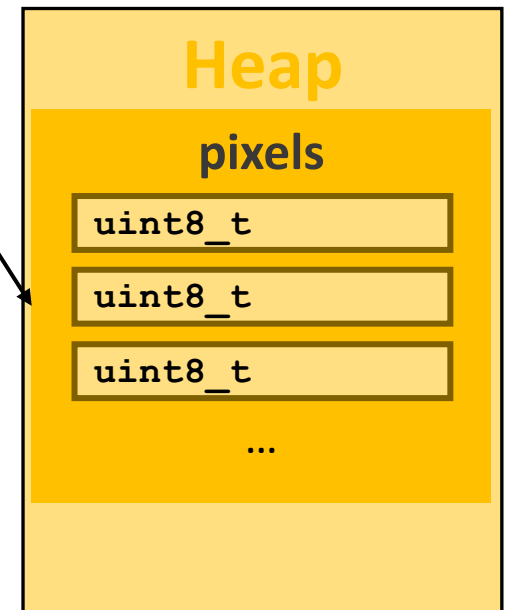
private:
    uint8_t* m_pixels; // Zeiger auf dynamisches Array
    size_t m_pixelCount; // Anzahl der Pixel im Array
};
```

Dynamische Speicherdauer



Dynamischer Prozessspeicher „Heap“
(Ausblick: Speicherverwaltung)

m_pixels



Kopiersemantik

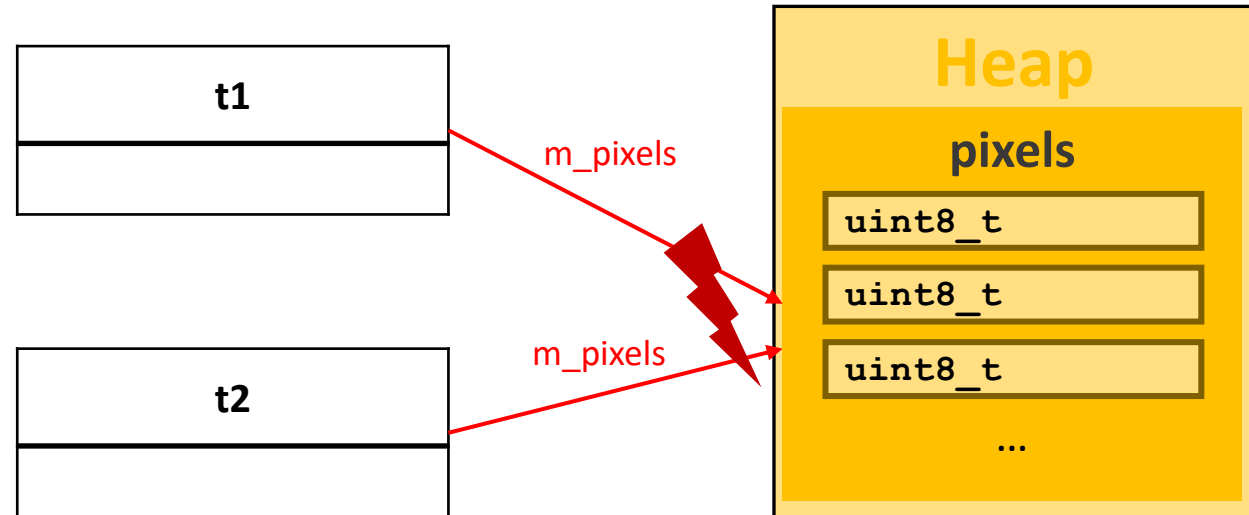


Kopien – Beispiel

- Wird das Texture-Objekt durch Compiler-generierte Methoden kopiert, wird lediglich die Zeigeradresse „m_pixels“ kopiert

→ **shallow copy**

```
int main()
{
    Texture t1;
    t1.loadFromFile("wall.png");
    Texture t2 { t1 };
}
```



- Es zeigen mehrere unterschiedliche Objekte auf den gleichen Speicherbereich
 - **Ungewollte Veränderung** eines fremden Objekts
 - **Heap corruption** durch doppelte Deallokation bei Objektzerstörung
(Ausblick: Speicherverwaltung)

Kopiersemantik



Kopien – Beispiel

- Gewollt ist eher, die Pixeldaten in das zweite Objekt zu kopieren
- Hierzu muss der Kopierkonstruktor explizit definiert werden:

```
class Texture
{
public:
    ...
    Texture(const Texture& other)
    {
        m_pixels = new uint8_t[other.m_pixelCount];
        memcpy(m_pixels, other.m_pixels, other.m_pixelCount);
        m_pixelCount = other.m_pixelCount;
    }
    ~Texture()
    {
        delete[] m_pixels;
    }

private:
    uint8_t* m_pixels;
    size_t m_pixelCount;
};
```

Explizit definierter
Kopierkonstruktor

Neuer Speicherbereich
für zu-kopierende Pixel

Kopie der Pixeldaten
in den neuen
Speicherbereich

→ Der Kopierkonstruktor führt
jetzt eine sogenannte „**deep
copy**“ (ausführliche Kopie) des
Objekts durch

Kopiersemantik

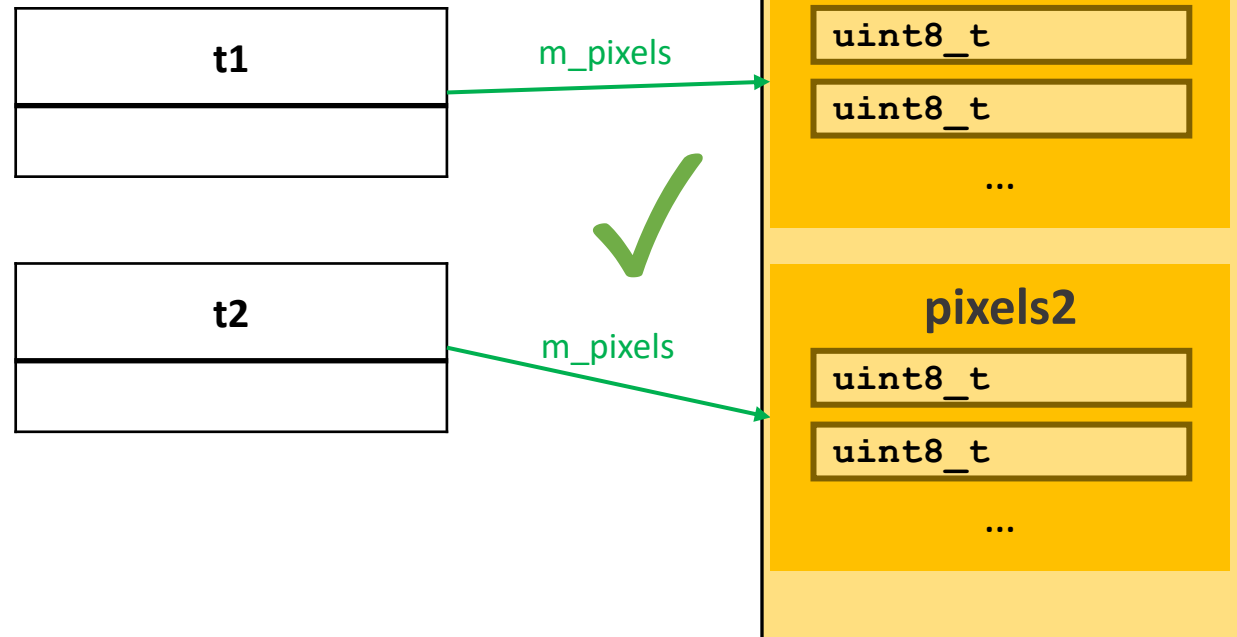
Kopien – Beispiel



- Nun wird das Texture-Objekt korrekt kopiert
 - Änderungen an einem Texture-Objekt beeinflussen das andere nicht
 - Speicherbereiche für die Pixeldaten werden nicht doppelt gelöscht

```
int main()
{
    Texture t1;
    t1.loadFromFile("wall.png");
    Texture t2 { t1 };
}
```

Aufruf des explizit definierten
Kopierkonstruktors, der deep
copy durchführt



Kopiersemantik



Zuweisung vs. Konstruktion

- Warum gibt es Konstruktoren *und* Zuweisungsoperatoren für die Kopie (und Verschiebung) von Objekten?
- Zuweisungsoperationen unterscheiden sich von der Objekterstellung dahin gehend, dass sie ein **vorhandenes Objekt überschreiben** müssen
 - Das eigene Objekt muss u.U. **zuerst „aufgeräumt“** werden, bevor es von einem anderen Objekt beschrieben werden kann
- Beispiel: Kopier-Zuweisung des Texture-Objekts

```
Texture& operator=(const Texture& other)
{
    delete[] m_pixels;

    m_pixels = new uint8_t[other.m_pixelCount];
    memcpy(m_pixels, other.m_pixels, other.m_pixelCount);
    m_pixelCount = other.m_pixelCount;

    return *this;
}
```

Vorheriger Speicherbereich für
Pixeldaten muss zuerst
freigegeben werden
→ Aufräumen

Danach kann Zeiger mit neuem
Speicherbereich sicher
überschrieben werden

Kopiersemantik

Selbstzuweisungen



- Bei Zuweisungsoperationen besteht zusätzlich die Gefahr, dass ein Objekt **auf sich selbst zugewiesen** wird
 - Dabei kommt es leicht zu Beschädigungen des Prozessspeichers (*Ausblick: Speicherverwaltung*)

```
int main()
{
    Texture t;
    t.loadFromFile("wall.png");

    t = t;
}
```

Zuweisung auf sich selbst;
Valide Syntax!

m_pixels ist jetzt gleich other.m_pixels

```
Texture& operator=(const Texture& other)
{
    delete[] m_pixels;

    m_pixels = new uint8_t[other.m_pixelCount];
    memcpy(m_pixels, other.m_pixels, other.m_pixelCount);
    m_pixelCount = other.m_pixelCount;

    return *this;
}
```



Beschädigungen am Prozessspeicher

Kopiersemantik

Selbstzuweisungen



- Eine mögliche Technik, um die Zuweisung auf sich selbst abzufangen, ist zu überprüfen, ob sich das zu-kopierende Objekt an der **gleichen Speicheradresse** befindet wie das eigene Objekt



```
Texture& operator=(const Texture& other)
{
    if (this != &other)
    {
        delete[] m_pixels;

        m_pixels = new uint8_t[other.m_pixelCount];
        memcpy(m_pixels, other.m_pixels, other.m_pixelCount);
        m_pixelCount = other.m_pixelCount;
    }

    return *this;
}
```

Check, ob anderes Objekt an gleicher Speicheradresse
→ Wenn ja: keine Kopieroperation

Referenz auf eigenes Objekt muss in jedem Fall zurückgegeben werden

Kopiersemantik

Zusammenfassung



- Wird ein Objekt einem anderen Objekt zugewiesen, so wird eine Objektinstanz **kopiert**
- Das Kopierverhalten von Objekten kann **gesteuert** werden
 - Ohne weitere Angaben werden die Membervariablen eines Objekts trivial kopiert (= „*shallow copy*“)
 - Durch Definition eines eigenen **Kopierkonstruktors** und **Kopier-Zuweisungsoperators** können die Kopieroperationen selbst definiert werden, z.B. für eine „*deep copy*“
- Es wird unterschieden ob ein **neues Objekt** bei seiner Erstellung zugewiesen wird oder ein **bereits vorhandenes Objekt** zu einem späteren Zeitpunkt (neu) zugewiesen wird
 - Bei der Erstellung behandelt der **Konstruktor** die Kopie
 - Bei der Zuweisung auf ein vorhandenes Objekt behandelt der **Zuweisungsoperator** die Kopie
 - Zuweisungsoperatoren müssen neben dem Kopier- oder Verschiebeverhalten:
 1. Ein bestehendes Objekt „**aufräumen**“
 2. Eine **Referenz auf die eigene Instanz** zurückgeben
 3. Sicherstellen, dass bei der **Selbstzuweisung** keine Probleme auftreten



Verschiebungssemantik

Lernziele

- Das Wissen über die Zuweisung von Objektinstanzen (Klassen) wird um die **Verschiebung von Objekten** erweitert
 - Es kann nachvollzogen werden, wann eine Objektinstanz **verschoben** wird und wie dieses Verhalten **gesteuert** werden kann
 - Dadurch wird nachvollzogen, warum und wie im Detail **Verschiebungssemantiken** durch moderne Sprachkonzepte eingeführt wurden
- Explizite Verschiebungsoperationen können für eigene Klassen definiert und optimiert werden

Verschiebungssemantik



Verschiebungen

- Wenn ein Objekt nicht mehr gebraucht wird und einem anderen Programmteil übergeben werden soll, ist es nützlich das Objekt zu verschieben, statt es zu kopieren
 - Der `C++11` Standard führt dazu die **Verschiebungssemantik** ein
 - Verschiebungen sind i.d.R. effizienter bei Objekten mit Verweisen (Zeigern) auf große Datenstrukturen
- Neben dem Kopierkonstruktor und Kopier-Zuweisungsoperator, wird ein **Verschiebungskonstruktor** und **Verschiebungs-Zuweisungsoperator** definiert
 - Diese definieren das Verschiebungsverhalten von Objekten
 - Sie werden analog zur Kopiersemantik ebenfalls vom Compiler implizit generiert und können explizit überschrieben werden

Verschiebungssemantik



Verschiebungen – Beispiel

- Beispiel: Ein **Texture**-Objekt wird einer rendernden Komponente über eine Setter-Funktion zugewiesen

```
class RenderTarget
{
public:
    void setTexture(Texture texture)
    {
        m_texture = texture;
    }

private:
    Texture m_texture;
};
```

```
int main()
{
    Texture t;
    t.loadFromFile("wall.png");

    RenderTarget rt;
    rt.setTexture(t);
    ...
}
```

Zuweisung durch
Setter-Methode

Schlechte Performance!
Warum?

Verschiebungssemantik

Verschiebungen – Beispiel



Es existieren drei Texture-Objekte und zwei Übergaben:

```
class RenderTarget
{
public:
    void setTexture(Texture texture)
    {
        m_texture = texture;
    }
private:
    Texture m_texture;
};
```

Kopie 1

Texture-Objekt u.U.
mehrere Megabyte groß

```
int main()
{
    Texture t;
    t.loadFromFile("wall.png");

    RenderTarget rt;
    rt.setTexture(t);
    ...
}
```

Kopie 2

→ Texture-Objekt wird samt
Pixeldaten zweimal kopiert
→ Verschiebung statt
Kopie sinnvoll

Verschiebungssemantik

Verschiebungskonstruktor



- Der **Verschiebungskonstruktor** erstellt die Instanz der Klasse als Verschiebung aus einer anderen Instanz der Klasse
 - Dazu erhält er ein Argument: Die **R-Value-Reference** auf die Instanz, aus der verschoben werden soll
 - Das referenzierte Objekt ist also temporärer Natur und darf bedenkenlos verändert werden!
- Der Compiler generiert automatisch einen Verschiebungskonstruktor, der jede einzelne Membervariable der Klasse verschiebt

```
class Texture
{
public:
    Texture() { m_pixels = nullptr; }
    Texture(const Texture& other) { ... }
    Texture(Texture&& other)
    {
        ...
    }
    ~Texture() { delete[] m_pixels; }

private:
    uint8_t* m_pixels;
};
```

Verschiebungskonstruktor

Verschiebungssemantik

Verschiebungs-Zuweisungsoperator

- Der **Verschiebungs-Zuweisungsoperator** verschiebt auf die eigene vorhandene Instanz (**this**) die Daten einer anderen Instanz, der gleichen Klasse
 - Dazu erhält er – wie der Verschiebungs-konstruktor – eine **R-Value-Reference** auf die Instanz, die zugewiesen & verschoben werden soll
- Der Compiler generiert automatisch einen Verschiebungs-Zuweisungsoperator für eine Klasse, der jede Membervariable verschiebt – wie beim Verschiebungs-konstruktor



```
class Texture
{
public:
    Texture() { m_pixels = nullptr; }
    Texture(const Texture& other) { ... }
    Texture(Texture&& other) { ... }
    Texture& operator=(const Texture& other) { ... }
    Texture& operator=(Texture&& other)
    {
        ...
    }
    ~Texture() { delete[] m_pixels; }

private:
    uint8_t* m_pixels;
};
```

Verschiebungs-
Zuweisungsoperator

Verschiebungssemantik

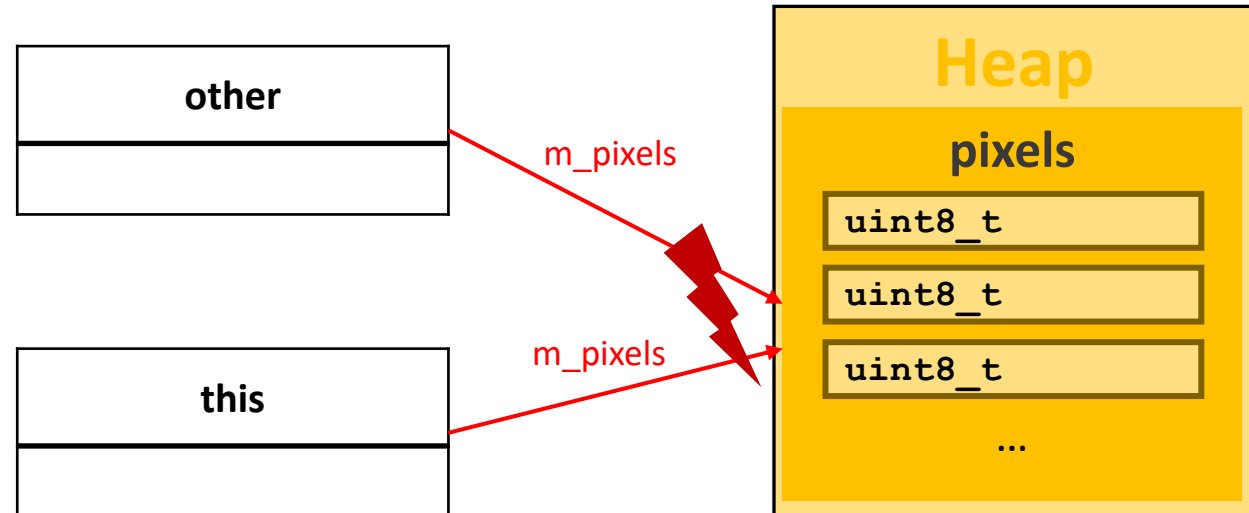


Verschiebungen – Beispiel

- Das Texture-Objekt kann sehr effizient verschoben werden, indem der Zeiger auf den dynamischen Speicherbereich der Pixeldaten übernommen wird

```
Texture(Texture&& other)
{
    m_pixels = other.m_pixels;
}
```

Die Kopie der Pixelanzahl (`m_pixelCount`) ist immer gleich und wird nachfolgend vernachlässigt



→ Es zeigen jedoch wieder mehrere unterschiedliche Objekte auf die gleichen Pixeldaten im Heap
→ Problem?

Verschiebungssemantik

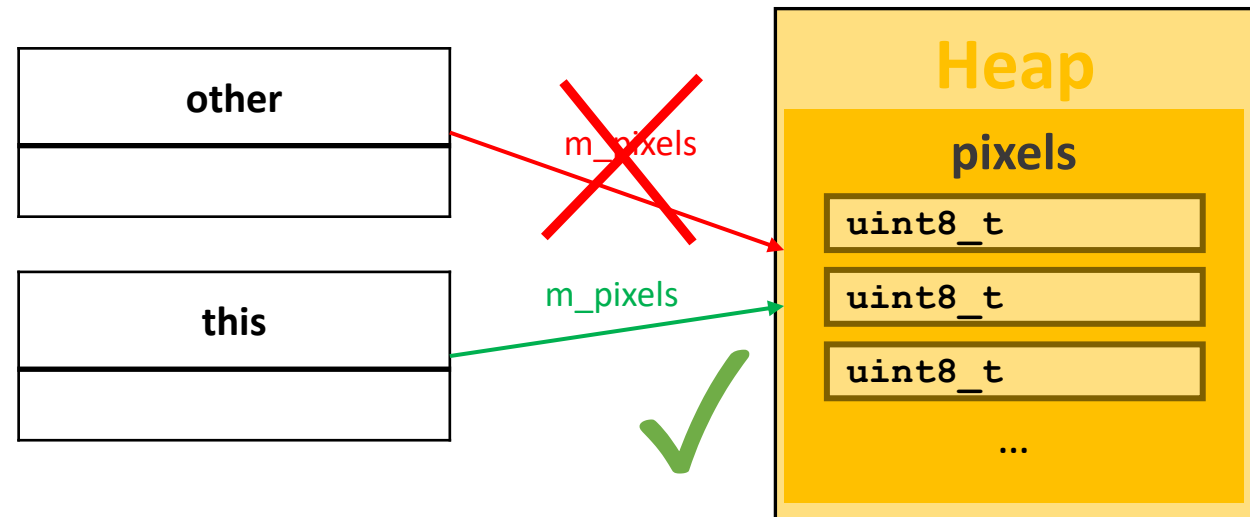


Verschiebungen – Beispiel

- Die **R-Value-Reference** garantiert, dass das referenzierte Objekt temporärer Natur ist und nicht weiter verwendet werden wird
- Es muss jedoch sichergestellt werden, dass das referenzierte Objekt **korrekt zerstört** wird!

→ Verschiebungskonstruktor muss Freigabe des dynamischen Speicherbereichs verhindern

```
Texture(Texture&& other)
{
    m_pixels = other.m_pixels;
    other.m_pixels = nullptr;
}
```



→ Nullsetzen des Zeigers im referenzierten Objekt verhindert Freigabe

Verschiebungssemantik



Verschiebungen – Beispiel

- Zurück zum Beispiel der Texture-Zuweisung zu einer rendernden Komponente...
- Wie kann angegeben werden, dass die Textur verschoben – statt kopiert – werden soll?

```
class RenderTarget
{
public:
    void setTexture(Texture texture)
    {
        m_texture = texture;
    }

private:
    Texture m_texture;
};
```

```
int main()
{
    Texture t;
    t.loadFromFile("wall.png");

    RenderTarget rt;
    rt.setTexture(t);
    ...
}
```

Zuweisung in
Setter-Funktion

Hier werden Kopien
durchgeführt
→ Verschiebung
erzwingen?

Verschiebungssemantik

move()-Methode



- Rückblick Referenzbindung:
 - Eine **non-modifiable L-Value-Reference** (`const T&`) – wie sie Kopierkonstrukten übergeben wird – bindet **konkrete und temporäre Objekte (L-Values, X-Values)**
 - Eine **R-Value-Reference** (`T&&`) – wie sie in Verschiebungskonstrukten übergeben wird – bindet **ausschließlich temporäre Objekte (über X-Values)**

→ Für den Aufruf des Verschiebungskonstruktors oder des Verschiebungs-Zuweisungsoperators, müssen Objekte entweder...

- als **PR-Value** vorliegen, oder...
- aus einem **L-Value in ein X-Value konvertiert** werden

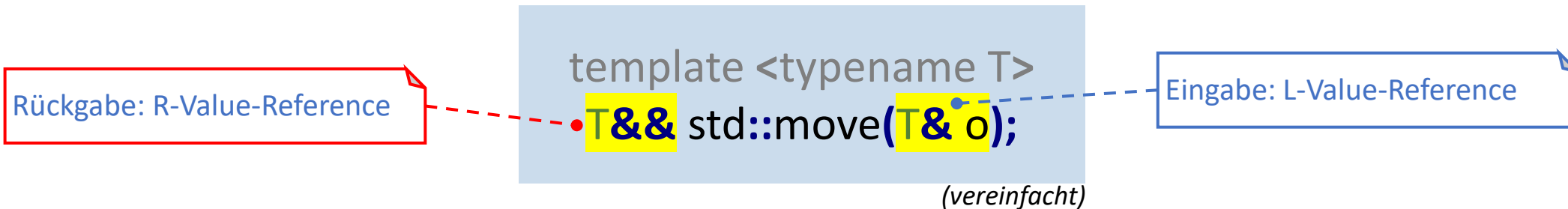
```
Number n1 = 1;  
Number n2 = n1; // führt Kopie aus  
// Wie kann hier Verschiebung erreicht werden?
```



Verschiebungssemantik

move()-Methode

- Die **std::move()** Methode konvertiert ein L-Value in ein X-Value
 - Die Methode allein führt noch keine Verschiebung durch!



- Das an die **std::move()** Methode übergebene Objekt wird nach deren Aufruf **ungültig** und sollte nicht weiter verwendet werden
 - Das konkrete Objekt (L-Value) wird zu einem vergänglichen Objekt (X-Value) konvertiert
 - Funktionen, die das X-Value erhalten, dürfen das Objekt verändern bzw. zerstören

```
Number n1 = 1;  
Number n2 = std::move(n1); // Verschiebung  
// n1 jetzt ungültig
```

Verschiebungssemantik



move()-Methode – Beispiel

- Durch `std::move()` kann angegeben werden, dass das Texture-Objekt in die rendernde Komponente verschoben – statt kopiert – werden soll

```
class RenderTarget
{
public:
    void setTexture(Texture texture)
    {
        m_texture = std::move(texture);
    }

private:
    Texture m_texture;
};
```

```
int main()
{
    Texture t;
    t.loadFromFile("wall.png");

    RenderTarget rt;
    rt.setTexture(std::move(t));
    ...
}
```

Doch was passiert hier genau?...

Verschiebungssemantik

move()-Methode – Beispiel



Ausgangssituation: Das konkrete Texture-Objekt „**Texture** t“ der Main-Funktion enthält eine vollwertige Textur mit Pixeldaten im Heap.

```
int main()
```

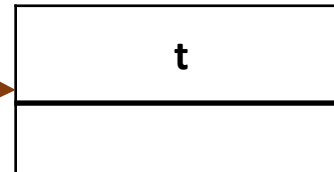
```
{
```

```
    Texture t;
```

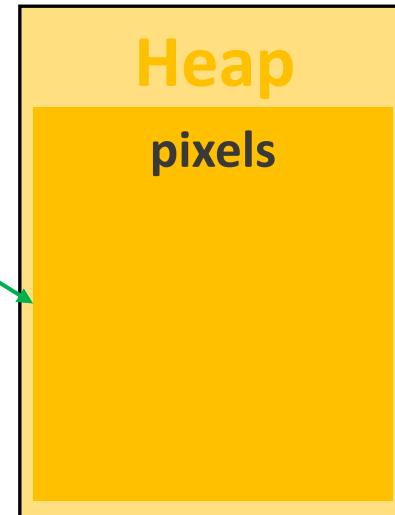
```
    t.loadFromFile("wall.png");
```

```
    ...
```

```
}
```



m_pixels



Verschiebungssemantik

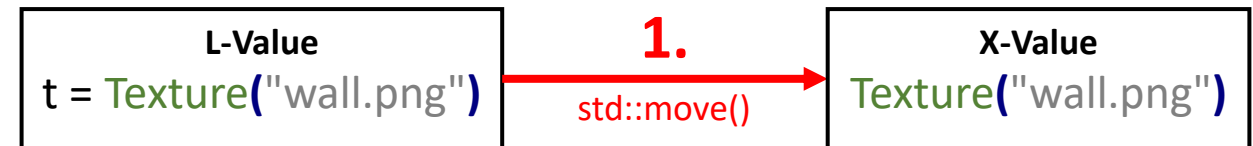
move()-Methode – Beispiel



1. Das konkrete Texture-Objekt „t“ (L-Value) wird durch **std::move()** gewandelt und als temporäres Objekt (X-Value) an die Setter-Funktion von „RenderTarget“ übergeben.

```
int main()
{
    Texture t;
    t.loadFromFile("wall.png");

    RenderTarget rt; 1.
    rt.setTexture(std::move(t));
    // t ab hier ungültig!
    ...
}
```



Noch ist das Texture-Objekt nicht verändert worden!
Es wurde lediglich ein neuer Wertausdruck darauf
erstellt (X-Value)

Per Definition von **std::move()** ist „t“ nach dem Aufruf ungültig und darf nicht weiter verwendet werden

Verschiebungssemantik



move()-Methode – Beispiel

2. Das Parameterobjekt „**Texture texture**“ (L-Value) der Setter-Funktion von „**RenderTarget**“ wird mit dem gewandelten Texture-Objekt „**t**“ (X-Value) *erstellt*.

→ Aufruf des Verschiebungsconstructors von „**Texture texture**“

```
int main()
{
    Texture t;
    t.loadFromFile("wall.png");

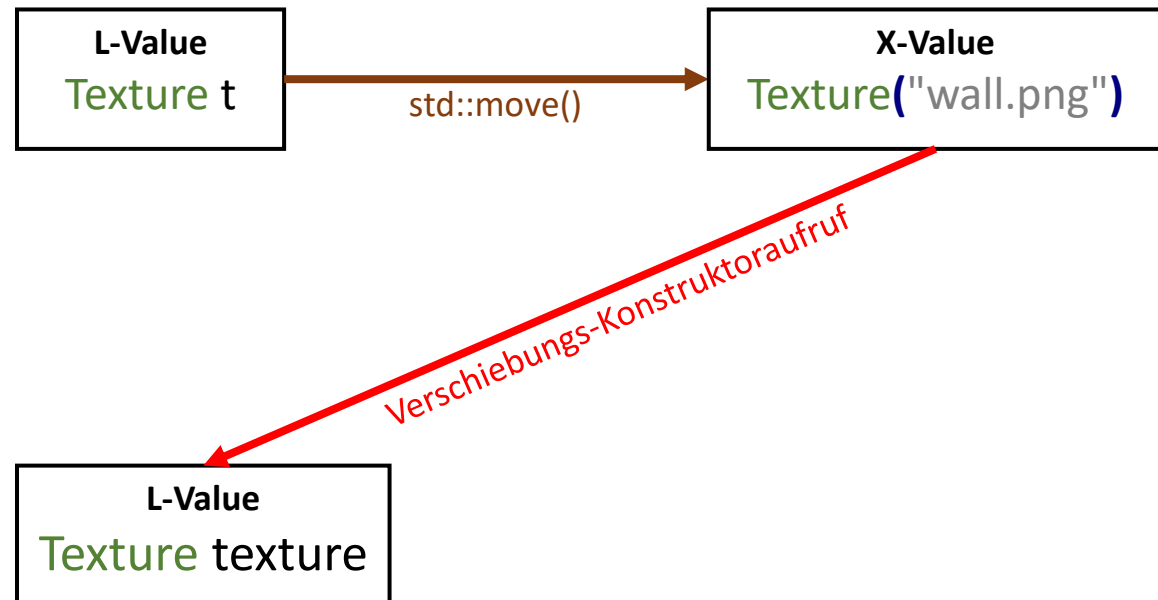
    RenderTarget rt;
    rt.setTexture(std::move(t));
    ...
}
```

```
class RenderTarget
{
public:
    void setTexture(Texture texture)
    { ... }
};
```

X-Value

2.

Parameterobjekt



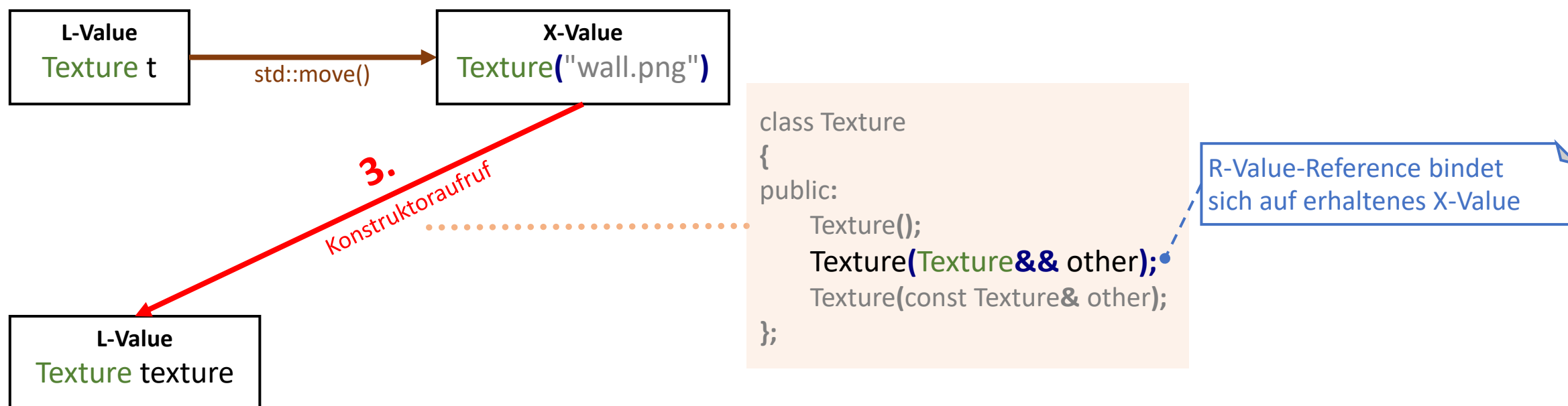


Verschiebungssemantik

move()-Methode – Beispiel

3. Da das gewandelte Texture-Objekt ein X-Value ist, wird der **Texture** **Verschiebungskonstruktor** mit dem R-Value-Reference Parameter aufgerufen.

Rückblick: X-Values werden *zuerst* von R-Value-References gebunden.



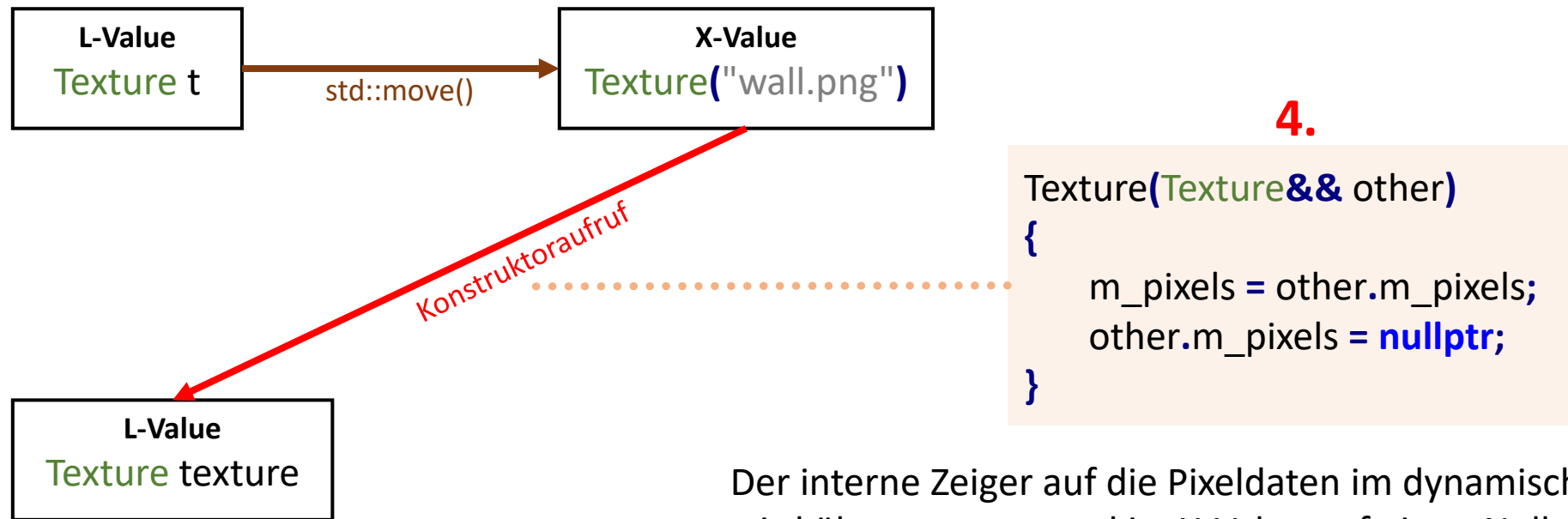
Verschiebungssemantik

move()-Methode – Beispiel



4. Der **Verschiebungsstruktur** übernimmt den dynamischen Speicherbereich der Pixeldaten des gewandelten Objekts „**other**“ (X-Value).

→ Damit führt er die **effektive Verschiebung** des Texture-Objekts in das Parameterobjekt „**texture**“ durch.



Der interne Zeiger auf die Pixeldaten im dynamischen Speicherbereich wird übernommen und im X-Value auf einen Nullpointer gesetzt

Verschiebungssemantik



move()-Methode – Beispiel

Zwischenergebnis: Das Parameterobjekt „**Texture texture**“ der Setter-Funktion enthält nun die vollwertige Textur

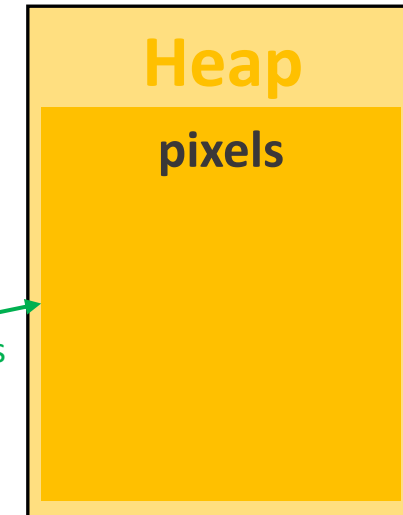
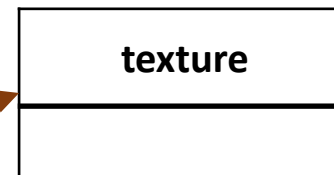
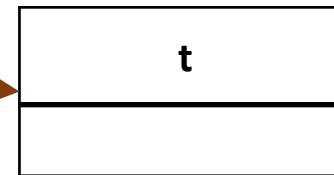
→ Das ursprüngliche Texture-Objekt „**t**“ ist „leer“ (und ungültig)

```
int main()
{
    Texture t;
    t.loadFromFile("wall.png");

    RenderTarget rt;
    rt.setTexture(std::move(t));
    ...
}
```

```
class RenderTarget
{
public:
    void setTexture(Texture texture)
    { ... }
};
```

Parameterobjekt



m_pixels

Bei diesem Vorgang wurde keine teure Kopie (Allokation + Deallokation) der Pixel durchgeführt!

Verschiebungssemantik

move()-Methode – Beispiel



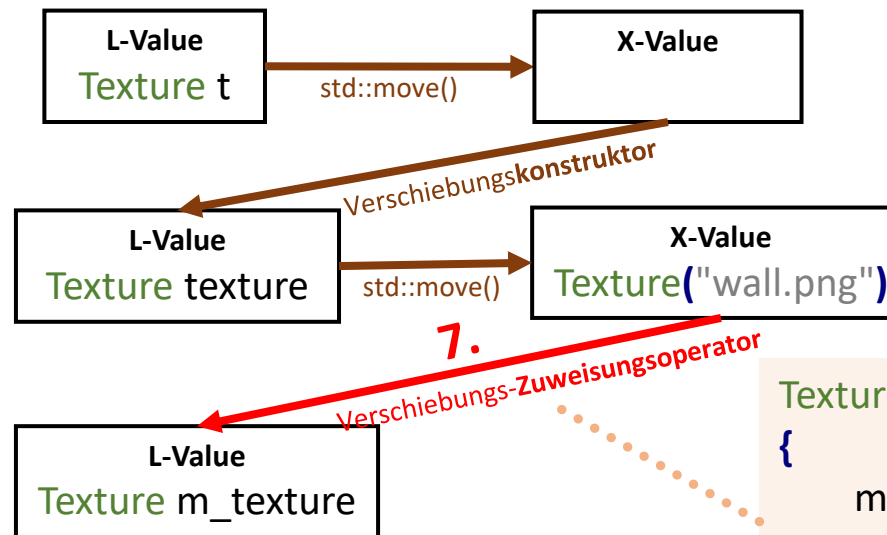
5. - 8. Für die Verschiebung des Parameterobjekts „**texture**“ in die Membervariable „**m_texture**“ wiederholt sich dieser Vorgang. Nun wird jedoch in ein bestehendes Objekt verschoben.

→ Verschiebungs-Zuweisungsoperator

```
class RenderTarget
{
public:
    void setTexture(Texture texture)
    {
        m_texture = std::move(texture);
    }

private:
    Texture m_texture;
};
```

„**texture**“ ist nach dem Aufruf ungültig und darf nicht weiter verwendet werden



```
Texture& operator=(Texture&& other)
{
    m_pixels = other.m_pixels;
    other.m_pixels = nullptr;
    return *this;
}
```

Verschiebungssemantik

move()-Methode – Beispiel



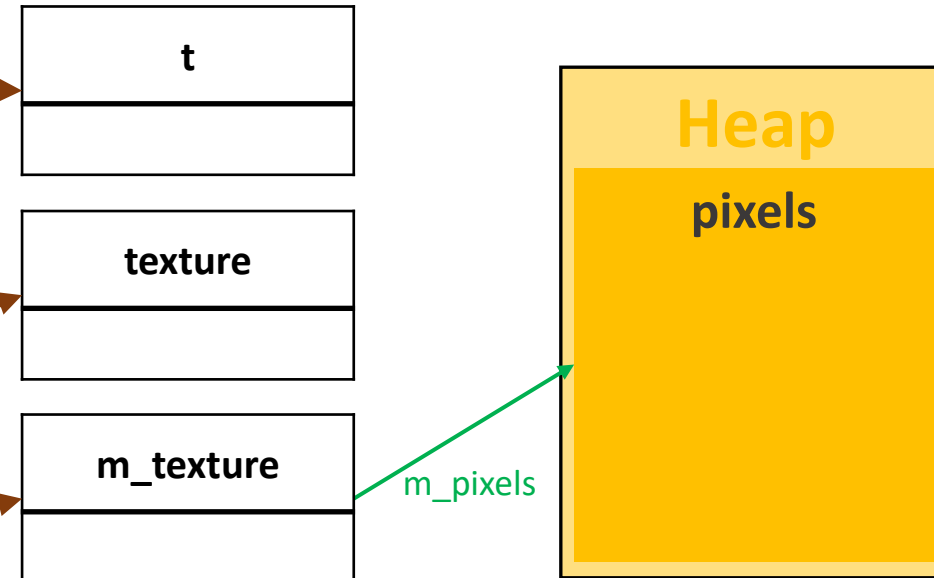
Endergebnis: Die Membervariable „**Texture** m_texture“ der **RenderTarget** Instanz enthält nun die vollwertige Textur

→ Das ursprüngliche Texturobjekt „t“ und das Parameterobjekt „texture“ ist „leer“

```
int main()
{
    Texture t;
    t.loadFromFile("wall.png");

    RenderTarget rt;
    rt.setTexture(std::move(t));
    ...
}

class RenderTarget
{
public:
    void setTexture(Texture texture)
    { m_texture = std::move(texture); }
private:
    Texture m_texture;
};
```



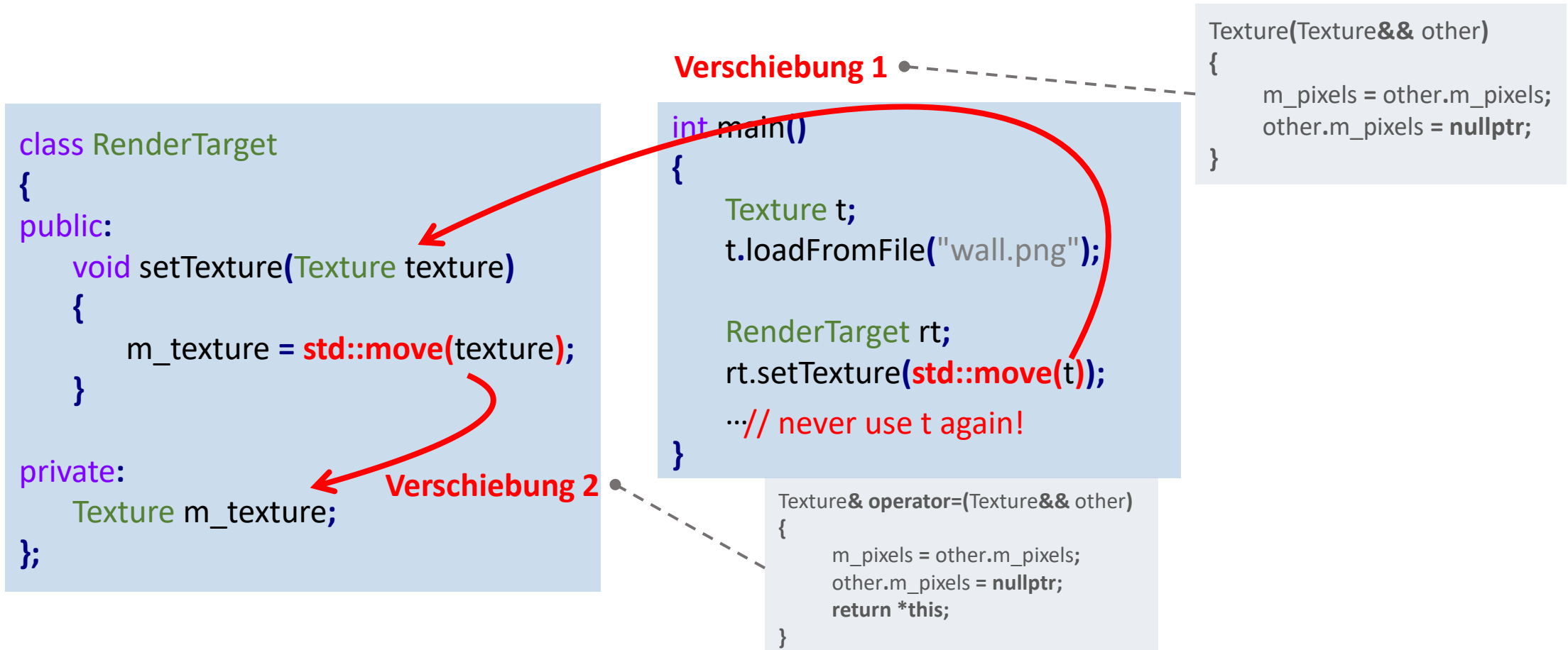
Beim gesamten Zusweisungsverfahren wurden **keine** teuren **Kopien** (Allokation + Deallokation) der Pixel durchgeführt!



Verschiebungssemantik

move()-Methode – Beispiel

- Semantisch betrachtet wurde das Texturobjekt zweimal verschoben

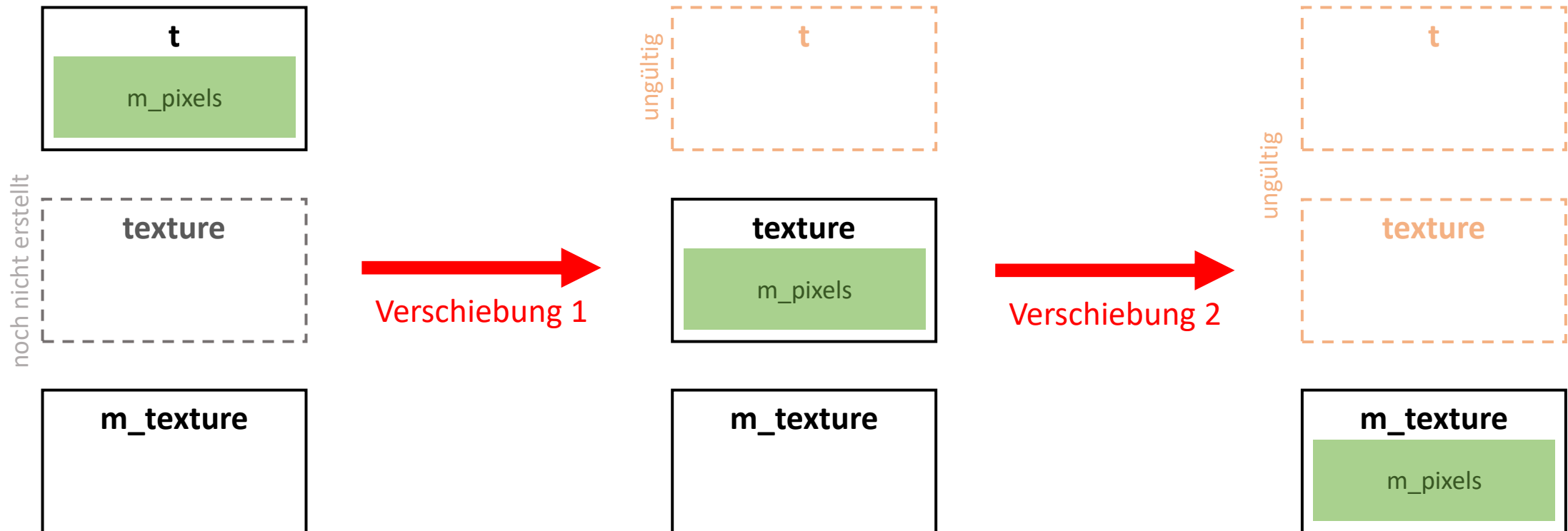


Verschiebungssemantik

move()-Methode – Beispiel



- Dabei können die konkreten Objekte (L-Values) sinnbildlich als **Hülle** betrachtet werden, die „ausgesaugt“ werden und deren Inhalt durchgereicht wird



Verschiebungssemantik



Zusammenfassung

- Wird ein temporäres Objekt (R-Value) einem konkreten Objekt (L-Value) zugewiesen, so wird versucht eine Objektinstanz zu **verschieben**
 - Dies wird seit **C++11** mit R-Value-References ermöglicht
- Das Verschiebungsverhalten von Objekten kann **gesteuert** werden
 - Ohne weitere Angaben werden die Membervariablen eines Objekts verschoben so weit möglich, sonst kopiert
 - Durch Definition eines eigenen **Verschiebungskonstruktors** und **Verschiebungs-Zuweisungsoperators** können die Verschiebungsoperationen selbst definiert werden
- Mit der **std::move()** Methode können konkrete Objekte (L-Values) verschiebbar gemacht werden (L-Value → X-Value)
 - Die eigentliche Verschiebung führt erst der Verschiebungskonstruktor oder -Zuweisungsoperator durch



Verständnisfragen

■ WAHR oder FALSCH?

- Der Kopierkonstruktor erzeugt ein Objekt als Kopie eines anderen Objekts gleichen Typs.
- Wird auf ein bestehendes Objekt ein (anderes) Objekt zugewiesen, wird ein Zuweisungsoperator ausgeführt.
- Ein Zuweisungsoperator kann exakt so implementiert werden, wie ein Konstruktor.
- Der Compiler führt implizit eine "deep copy" aus, wenn kein eigener Kopierkonstruktor angegeben wurde.
- Die Zuweisung eines Objekts auf sich selbst stellt keine Probleme dar.
- Die Verschiebung eines Objekts kann billiger sein, als die Kopie eines Objekts.
- Die Funktion `std::move()` verschiebt die Daten eines Objekts.

```
int main()
{
    Number n;
    n = 42;
}
```

■ Welche Operationen werden hier ausgeführt?

- | | |
|--|---|
| ■ <code>Number::Number()</code> | ■ <code>Number::Number(Number&&)</code> |
| ■ <code>Number::Number(int n)</code> | ■ <code>Number::operator=(const Number&)</code> |
| ■ <code>Number::Number(const Number&)</code> | ■ <code>Number::operator=(Number&&)</code> |



Compiler-generierte Konstrukte

Lernziele

- Es wurde verstanden wann der Compiler welche Konstruktoren, Zuweisungsoperatoren und Destruktoren erstellt
 - Dazu wird gezeigt, welche Auswirkung die Definition eigener Konstrukte auf das Standardverhalten hat
- Das **Kopier**-, **Verschiebe**- und **Lösch**verhalten von Objekten kann anhand der vorhandenen Methoden einer Klasse nachvollzogen werden

Compiler-generierte Konstrukte

Implizite Konstrukte



Vom Programmierer definiert:

Vom Compiler generiert:		nichts	ctor	dtor	Copy-ctor	Copy-op=	Move-ctor	Move-op=
	ctor	✓	✓	✓	✗	✓	✗	✓
	dtor	✓	✓	✓	✓	✓	✓	✓
	Copy-ctor	✓	✓	✓	.	✓	✗	✗
	Copy-op=	✓	✓	✓	✓	.	✗	✗
	Move-ctor	✓	✓	✗	Verschiebung wird zu Kopie		.	✗
	Move-op=	✓	✓	✗			✗	.

ctor: Standardkonstruktor

dtor : Standarddestruktor

Copy-ctor : Kopier-Konstruktor

Copy-op= : Kopier-Zuweisungsoperator

Move-ctor : Verschiebungs-Konstruktor

Move-op= : Verschiebungs-Zuweisungsoperator

(Basierend auf: <https://blog.feabhas.com/2015/11/becoming-a-rule-of-zero-hero/>)

Compiler-generierte Konstrukte

Standardkonstruktor

- Rückblick Objektorientierung: Der Compiler generiert einen **parameterlosen Konstruktor** (Standardkonstruktor), der die Konstruktoren der Membervariablen aufruft
- Wird ein eigener **expliziter Konstruktor** definiert, erzeugt der Compiler keinen parameterlosen Konstruktor mehr
 - Eigene Konstruktoren werden weiter um die Konstruktoraufrufe der Membervariablen erweitert
 - Kopier- und Verschiebungskonstruktoren sind explizite Konstruktoren und deaktivieren damit den Standardkonstruktor!



```
class Texture { };

int main()
{
    ✓ Texture t; // Ok: Standardkonstruktor
}
```

```
class Texture
{
public:
    Texture(const Texture&) { ... }
};

int main()
{
    ⚡ Texture t; // Compilerfehler
}
```

Eigener **Kopierkonstruktor**
deaktiviert
Standardkonstruktor

Compiler-generierte Konstrukte

Konstruktoren & Zuweisungsoperatoren

- Der Compiler generiert **implizite Konstruktoren** und **Zuweisungsoperatoren** einer Klasse unter bestimmten Bedingungen
 - Wenn keine expliziten Kopier- oder Verschiebungsstrukture existieren, erstellt diese der Compiler
 - Wenn der Programmierer **explizit** einen Verschiebungs-Konstruktor bzw. einen Verschiebungs-Zuweisungsoperator **angibt**, wird kein Kopier-Konstruktor oder Kopier-Zuweisungsoperator generiert
 - Der Compiler geht davon aus, dass das Objekt dann nur verschoben werden kann
 - Wenn mindestens eine Membervariable nicht zugewiesen werden kann (const), werden keine Zuweisungsoperatoren generiert



```
class Texture { };

int main()
{
    Texture t1;
    ✓ Texture t2 { t1 }; // Ok: Kopierkonstruktor
    ✓ Texture t3 { std::move(t1) }; // Ok:
    Verschiebungskonstruktor
}
```

```
class Texture
{
private:
    const size_t m_pixelCount;
};

int main()
{
    Texture t1, t2;
    ⚡ t2 = t1; // Compilerfehler
}
```

Konstante als
Membervariable deaktiviert
Kopier-Zuweisungsoperator

Compiler-generierte Konstrukte

Standarddestruktor

- Der Compiler generiert einen **impliziten Destruktor** (Standarddestruktor), der die Destrukturen der Membervariablen aufruft
- Wird ein eigener **expliziter Destruktor** definiert, erweitert der Compiler diesen um die Destruktoraufrufe der Membervariablen
 - Der Aufruf der Konstruktoren und Destrukturen von Membervariablen ist immer sichergestellt



```
class Texture
{
    ~Texture() •----- Standarddestruktor
    {
        m_name.~string();
    }
    std::string m_name;
};

int main()
{
    Texture t;
    ✓ } // Ok: String von Standarddestruktor zerstört
```

Destruktoraufruf Membervariable

Compiler-generierte Konstrukte



Implizite Konstrukte Löschen / Erzwingen

- Seit **C++11** können Compiler-generierte Konstrukte auch **explizit „gelöscht“** werden
 - Hierzu wird der Konstruktor, Zuweisungsoperator oder Destruktor in der **Klassendefinition** gleich dem Keyword „**delete**“ gesetzt
 - Das verhindert die implizite Generierung durch den Compiler
- Auch kann seit **C++11** die Erzeugung dieser Konstrukte **explizit erzwungen** werden
 - Hierzu wird der Konstruktor, Zuweisungsoperator oder Destruktor in der **Klassendefinition** gleich dem Keyword „**default**“ gesetzt
 - Das zwingt den Compiler zum Erstellen der Konstrukte, die die Kopier- bzw. Verschiebungs-Operation auf jeden Member anwenden bzw. die Destruktoren aufruft
- Die Generierung explizit zu erzwingen oder zu verhindern, dient oft der eindeutigen Deklaration der Kopier- bzw. Verschiebbarkeit von Objekten
 - Nur in wenigen Fällen ist der Eingriff technisch notwendig

Compiler-generierte Konstrukte



Implizite Konstrukte Löschen / Erzwingen – Beispiel

- Beispiel 1: Eine Klasse erzwingt die Erzeugung des Standardkonstruktors

```
class TextureStore
{
public:
    TextureStore() = default;
    TextureStore(Texture* textureArray, size_t arraySize)
        : m_textures(textureArray, textureArray + arraySize)
    {}

private:
    std::vector<Texture> m_textures;
};
```

Erzwingt Erzeugung des parameterlosen Konstruktors
(Compiler generiert Konstruktor für „m_textures“ Member)

```
int main()
{
    // default-Konstruktor
    ✓ TextureStore store1;

    Texture textures[2];
    textures[0].loadFromFile("Wall.png");
    textures[1].loadFromFile("Floor.png");

    // expliziter Konstruktor
    ✓ TextureStore store2 { textures, 2 };
}
```

Compiler-generierte Konstrukte



Implizite Konstrukte Löschen / Erzwingen – Beispiel

- Beispiel 2: Auf Grund von Performanzoptimierungen, sollen Texture-Objekte nur verschoben, nicht kopiert werden dürfen

```
class Texture
{
public:
    Texture() { m_pixels = nullptr; }
    Texture(const Texture& other) = delete;
    Texture(Texture&& other) { ... }
    Texture& operator=(const Texture& other) = delete;
    Texture& operator=(Texture&& other) { ... }
    ~Texture() { delete[] m_pixels; }

private:
    uint8_t* m_pixels;
};
```

Löschung des Kopierkonstruktors
und Kopier-Zuweisungsoperators

```
int main()
{
    Texture t1;
    t1.loadFromFile("wall.png");

    ✖ Texture t2 { t1 }; // Compilerfehler
    ✔ Texture t3 { std::move(t1) }; // Ok
}
```

(Technisch hätte hier auch die alleinige
Deklaration der Verschiebungsstrukture genügt)

Compiler-generierte Konstrukte



Probleme

- Welche Konstruktoren und Zuweisungsoperatoren für eine Klasse zur Verfügung stehen ist nicht immer klar ersichtlich, wenn diese nicht explizit angegeben sind
→ Schwierig auf den ersten Blick nachzuvollziehen, wie Objekte einer Klasse verwendet werden können

```
class Texture
{
};
```

✓ ctor (parameterlos)
✓ copy-ctor
✓ move-ctor
✓ copy-op=
✓ move-op=
✓ dtor

```
class Texture
{
public:
    Texture(const Texture&);
};
```

✗ ctor (parameterlos)
✓ copy-ctor
~ move-ctor (ruft copy auf)
✓ copy-op=
~ move-op= (ruft copy auf)
✓ dtor

```
class Texture
{
public:
    Texture& operator=(Texture&&);
};
```

✓ ctor (parameterlos)
✗ copy-ctor
✗ move-ctor
✗ copy-op=
✓ move-op=
✓ dtor

```
class Texture
{
public:
    ~Texture();
};
```

✓ ctor (parameterlos)
✓ copy-ctor
✗ move-ctor
✓ copy-op=
✗ move-op=
✓ dtor

Compiler-generierte Konstrukte



Rule of 5

- Häufig müssen bei der Behandlung eines Kopier-, Verschiebungs- oder Zerstörungsfalls auch die jeweils anderen Fälle behandelt werden
 - Z.B. Verwaltung von Objekten mit dynamischer Speicherdauer (z.B. Pixel der **Texture**-Klasse)
- Die „**Rule of 5**“ besagt daher:

Wenn eine Klasse mindestens eine der folgenden Konstrukte definiert, muss sie alle 5 dieser Konstrukte definieren:

- 1 **Kopierkonstruktor**
- 2 **Kopier-Zuweisungsoperator**
- 3 **Verschiebungskonstruktor**
- 4 **Verschiebungs-Zuweisungsoperator**
- 5 **Destruktor**

Vor der Einführung von Verschiebungsoperationen durch den **C++11** Standard, hieß die Regel „**Rule of 3**“

Compiler-generierte Konstrukte



Rule of 5

- Durch die explizite Definition aller Kopie-, Verschiebungs- und Zerstörungsoperationen, wird die **beabsichtigte Nutzung** der Klasse **eindeutig**

```
class Texture
{
public:
    Texture();
    Texture(const Texture&);
    Texture(Texture&&);
    Texture& operator=(const Texture&);
    Texture& operator=(Texture&&);
    ~Texture();
};
```

- ✓ Parameterlose Konstruktion
- ✓ Kopie
- ✓ Verschiebung

```
class Texture
{
public:
    Texture();
    Texture(const Texture&) = delete;
    Texture(Texture&&);
    Texture& operator=(const Texture&) = delete;
    Texture& operator=(Texture&&);
    ~Texture();
};
```

- ✓ Parameterlose Konstruktion
- ✗ Kopie
- ✓ Verschiebung

```
class Texture
{
public:
    Texture();
    Texture(const Texture&);
    Texture(Texture&&) = delete;
    Texture& operator=(const Texture&);
    Texture& operator=(Texture&&) = delete;
    ~Texture();
};
```

- ✓ Parameterlose Konstruktion
- ✓ Kopie
- ✗ Verschiebung

Hinweis: Der parameterlose Konstruktor (Standardkonstruktor) gehört nicht zur Rule-of-5

Compiler-generierte Konstrukte



Rule of 0

- Verwendet eine Klasse keine Zeiger oder andere Konstrukte, die manuelle Operationen zur Kopie, Verschiebung oder Zerstörung benötigten, so sollte auf die Definition eigener Konstruktoren, Zuweisungsoperatoren und Destruktoren vollständig verzichtet werden
- Die „**Rule of 0**“ besagt daher:

Eine Klasse soll keinen Kopierkonstruktor, Kopier-Zuweisungsoperator, Verschiebungskonstruktor, Verschiebungs-Zuweisungsoperator oder Destruktor explizit definieren

Compiler-generierte Konstrukte



Rule of 0

- Klassen, deren Member alle der Rule-of-5 (oder Rule-of-0) folgen, sollten immer der Rule-of-0 folgen

Klasse nach Rule-of-5

```
class Texture
{
public:
    Texture();
    Texture(const Texture&) = delete;
    Texture(Texture&&);
    Texture& operator=(const Texture&) = delete;
    Texture& operator=(Texture&&);
    ~Texture();
};
```

- ✓ Parameterlose Konstruktion
- ✗ Kopie
- ✓ Verschiebung

Klasse nach Rule-of-0

```
class Image
{
public:
    Image(Texture texture)
        : m_texture{ std::move(texture) }
    {}

private:
    Texture m_texture;
};
```

Expliziter Konstruktor, aber kein Kopier-/Verschiebungskonstruktor!
→ Zählt nicht zur Rule-of-5 oder Rule-of-0

Verwendet Member, der Rule-of-5 folgt
→ Benötigt selbst keine Kopier-/Verschiebungskonstrukte oder Destruktor
(Diese werden vom Compiler implizit aufgerufen)

Compiler-generierte Konstrukte



Zusammenfassung

- Der **Compiler generiert** bestimmte Konstruktoren, Zuweisungsoperatoren und Destruktoren, wenn diese nicht vom Programmierer explizit angegeben werden
 - Die Generierung ist von bestimmten Bedingungen abhängig und häufig undurchsichtig
- Ein **manueller Eingriff** in die Erzeugung Compiler-generierter Konstrukte ist mit den Keywords „**delete**“ und „**default**“ möglich (**C++11**)
- Nach der **Rule-of-5** soll eine Klasse immer alle Kopier-, Verschiebungsoperationen und den Destruktor explizit definieren wenn mindestens eine dieser Operationen benötigt wird
 - Dabei können diese Operationen auch **delete** oder **default** gesetzt werden
- Nach der **Rule-of-0** soll eine Klasse keine Kopier-, Verschiebungsoperationen oder den Destruktor explizit definieren
 - Stattdessen sollen alle Membervariablen der Rule-of-5 bzw. Rule-of-0 folgen und damit dem Compiler die impliziten Aufrufe von Konstruktoren, Zuweisungsoperatoren und Destruktoren ermöglichen



Ausblick: Copy-and-swap

Lernziele

- Das Kopieren und Verschieben von Objekten mit der „**copy-and-swap**“ Technik wurde verstanden
 - Vorteile und Nachteile der Technik gegenüber bisher gelernten Methoden sind bekannt
 - *Copy-and-swap* kann für eigene Objekte implementiert werden

Copy-and-swap



Motivation

- Rückblick: Die **Selbstzuweisung** ist bei der Implementierung von Zuweisungsoperatoren ein Problem

```
int main()
{
    Texture t;
    t.loadFromFile("wall.png");

    t = t;
}
```

Zuweisung auf sich selbst;
Valide Syntax!

m_pixels ist jetzt gleich other.m_pixels

```
Texture& operator=(const Texture& other)
{
    delete[] m_pixels;

    m_pixels = new uint8_t[other.m_pixelCount];
    memcpy(m_pixels, other.m_pixels, other.m_pixelCount);
    m_pixelCount = other.m_pixelCount;

    return *this;
}
```



Beschädigungen am Prozessspeicher

- Weiter ist der Code der Zuweisungsoperatoren und der Konstruktoren oft sehr ähnlich bis gleich
→ doppelte Schreibaarbeit, Wartungsaufwand, Redundanz, ...

Copy-and-swap

Implementierung



- Das „**copy-and-swap**“ Idiom ist eine Programmieretechnik zur Implementierung von Zuweisungsoperatoren
 1. Ein **neues lokales Objekt** wird erzeugt und mit der zu-kopierenden Instanz initialisiert
→ Kopier- oder Verschiebungs-Konstruktoraufruf
 2. Das lokale Objekt wird mit der eigenen Instanz getauscht
→ Swap-Funktion
 3. Das lokale Objekt der Kopie wird zerstört
→ Destruktor

```
Texture& operator=(const Texture& other) • - - - - - Kopier-Zuweisungsoperator
{
    Texture copy(other); // 1. lokale Kopie

    copy.swap(*this); // 2. Tausch (swap)

    return *this;
} // 3. Zerstörung lokale Kopie
```

Copy-and-swap

Implementierung



- Das *copy-and-swap* Idiom macht sich den **Konstruktor** und **Destruktor** zu Nutze, um ein Objekt zuzuweisen

Erzeugung von „copy“

```
Texture& operator=(const Texture& other)
{
    Texture copy(other);
    copy.swap(*this);
    return *this;
}
```

```
Texture(const Texture& other)
{
    m_pixels = new uint8_t[other.m_pixelCount];
    memcpy(m_pixels, other.m_pixels, other.m_pixelCount);
    m_pixelCount = other.m_pixelCount;
}
```

Zerstörung von „copy“

```
~Texture()
{
    delete[] m_pixels;
}
```

Copy-and-swap

Swap-Funktion



- Für das Idiom muss eine **Swap-Funktion** definiert werden
 - Sie tauscht (*swap*) die Membervariablen einer Objektinstanz mit der einer anderen
 - Damit tauscht sie effektiv den Inhalt zweier Objekte (L-Values)

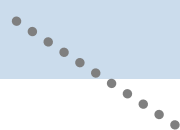
Tausch der Membervariablen

```
void swap(Texture& lhs, Texture& rhs)
{
    uint8_t* temp1 = lhs.m_pixels;
    lhs.m_pixels = rhs.m_pixels;
    rhs.m_pixels = temp1;

    size_t temp2 = lhs.m_pixelCount;
    lhs.m_pixelCount = rhs.m_pixelCount;
    rhs.m_pixelCount = temp2;
}
```

Schöner mit std::swap()

```
void swap(Texture& lhs, Texture& rhs)
{
    std::swap(lhs.m_pixels, rhs.m_pixels);
    std::swap(lhs.m_pixelCount, rhs.m_pixelCount);
}
```



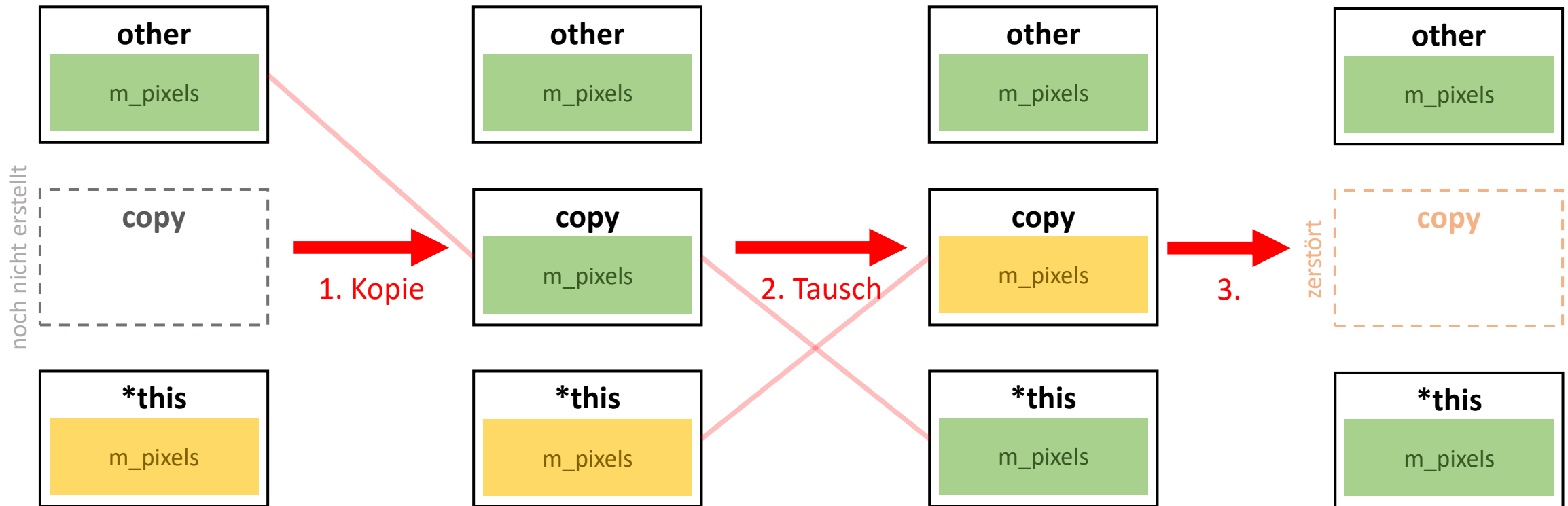
```
template <typename T>
void std::swap(T& left, T& right)
{
    T temp = left;
    left = right;
    right = temp;
}
```

Copy-and-swap

Veranschaulichung



- Kopie durch *copy-and-swap*:



Copy-and-swap



Verschiebung

- Auch die Zuweisung durch **Verschiebung** kann mit *copy-and-swap* umgesetzt werden

```
Texture& operator=(Texture&& other) •-----Verschiebungs-Zuweisungsoperator
{
    Texture copy(std::move(other)); // 1. lokale Kopie

    copy.swap(*this); // 2. Tausch (swap)

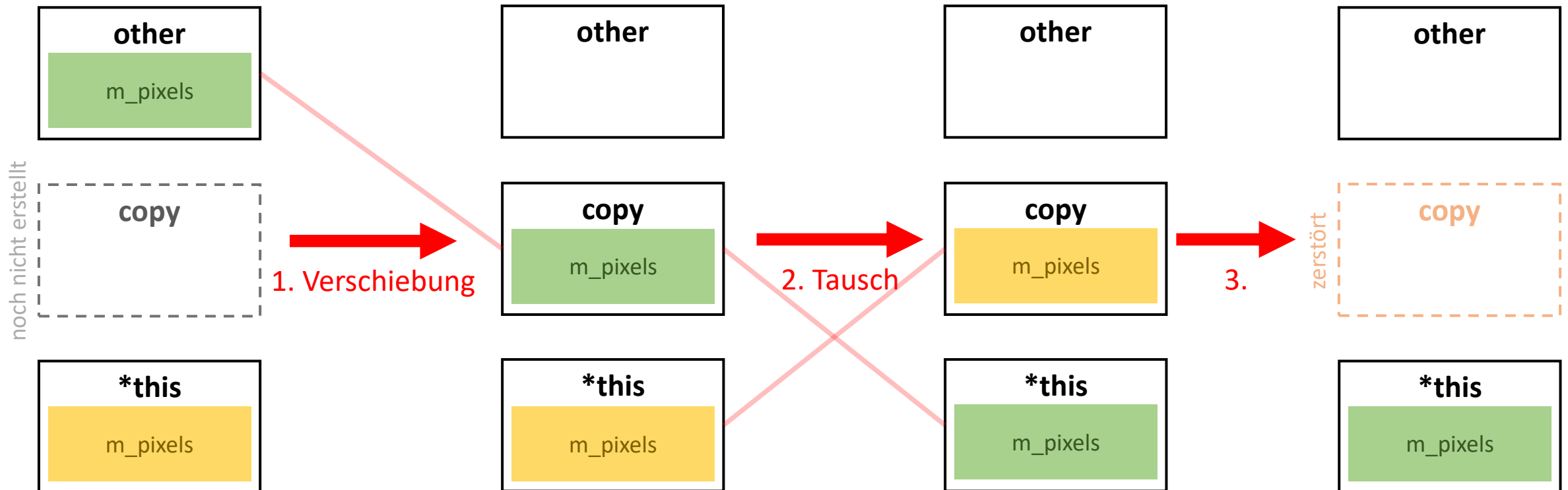
    return *this;
} // 3. Zerstörung lokale Kopie
```

Jetzt std::move(), damit
Verschiebungs-konstruktor aufgerufen wird!
Einzig Änderung!

Copy-and-swap

Verschiebung – Veranschaulichung

- Verschiebung durch *copy-and-swap*:



Copy-and-swap



Vorteile

- *Copy-and-swap* löst das Problem der **Selbstzuweisung**
 - Abfrage der Objektadresse ist nicht mehr nötig, da bei jeder Zuweisung ein neues Objekt erstellt wird

```
int main()
{
    Texture t;
    t.loadFromFile("wall.png");

    t = t;
}
```

Zuweisung auf sich selbst;
Valide Syntax!

Annahme: `&other == this`

```
Texture& operator=(const Texture& other)
{
    Texture copy(other); // 1. lokale Kopie
    copy.swap(*this); // 2. Tausch (swap)
    return *this;
} // 3. Zerstörung lokale Kopie
```

Hier entsteht neues Objekt!

Tausch mit neuem Objekt

Neues Objekt wird
gelöscht; nicht das eigene!

✓ keine Speicherverletzungen

Copy-and-swap

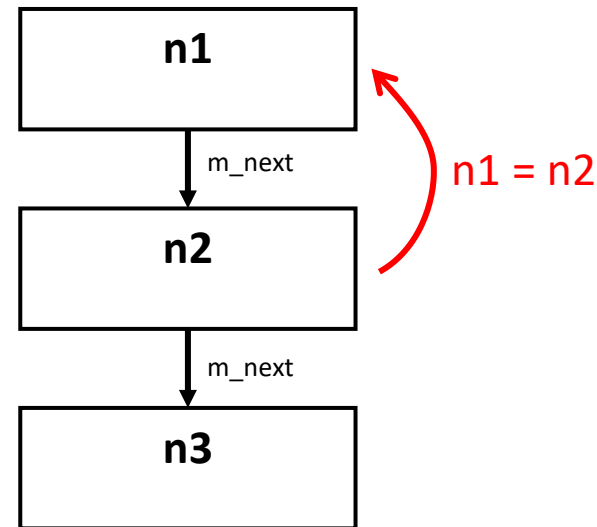
Vorteile

- *Copy-and-swap* funktioniert auch bei **rekursiven Datenstrukturen**
 - Diese sind häufig problematisch bei trivialen Zuweisungsoperatoren

```
class Node
{
    ...
    Node* m_next = nullptr;
};

int main()
{
    Node* n1 = new Node;
    Node* n2 = new Node;
    n1->m_next = n2;
    Node* n3 = new Node;
    n2->m_next = n3;

    *n1 = *n2;
}
```



Trivialer Zuweisungsoperator

```
Node& operator=(const Node& other)
{
    if (this != &other)
    {
        delete m_next;
        m_next = new Node;
        *m_next = *other.m_next;
    }

    return *this;
}
```

Löscht Teilbaum, der folgend kopiert werden soll

Enthält gelöschten Speicher

⚡ Beschädigungen am Prozessspeicher;
trotz Abfrage der Objektadresse!

Copy-and-swap

Vorteile & Nachteile



■ Weitere **Vorteile** von *copy-and-swap*:

- **Code-Wiederverwendung:** Kein doppelter Code für Kopierkonstruktion und Kopierzuzuweisung
- **Objektintegrität:** Schlägt die Kopieroperation fehl, bleibt das eigene (und das zu-kopierende) Objekt unbeschädigt

■ **Nachteile** von *copy-and-swap*:

- **Keine Speicherwiederverwendung:** Es entsteht immer ein neues Objekt; Vorhandener Speicher (z.B. Array eines Vektors) kann nicht wiederverwendet werden
- **Benötigt explizite Swap-Funktion:** Neben den Konstruktoren und Zuweisungsoperatoren wird eine Swap-Funktion benötigt, die alle Membervariablen einzeln tauschen muss

Copy-and-swap

Zusammenfassung



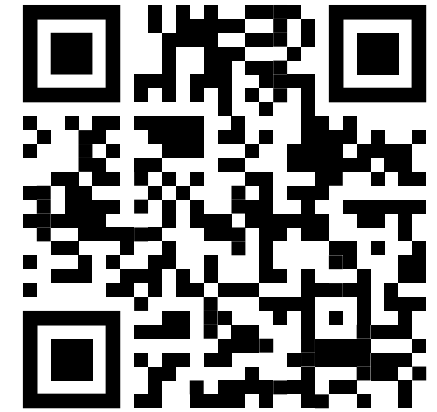
- Bei der „**copy-and-swap**“ Technik wird zuerst eine lokale Kopie des zu-kopierenden Objekts erstellt, die dann mit der eigenen Instanz getauscht wird
 - Die Technik bezieht sich auf Zuweisungsoperationen auf vorhandene Objekte
 - Die Technik macht sich den Kopier- bzw. Verschiebungskonstruktor und den **Destruktor** zu Nutze
 - Die Technik löst ohne weitere Abfragen das Problem der Selbstzuweisung
- Zur Umsetzung muss neben Konstruktoren, Zuweisungsoperatoren und dem Destruktor zusätzlich eine **Swap-Funktion** implementiert werden

Compiler-generierte Konstrukte & Copy-and-swap

Verständnisfragen

PIN: 327203

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Besitzt eine Klasse eine konstante Membervariable, erzeugt der Compiler keinen Zuweisungsoperator mehr.
- Wird ein eigener Destruktor definiert, muss dieser auch die Destrukturen der Membervariablen aufrufen.
- Die Rule-of-0 kann nur angewendet werden, wenn alle Membervariablen einer Klasse der Rule-of-5 oder Rule-of-0 folgen.
- Copy-and-swap ist eine Technik zur Initialisierung von Objekten.
- Die copy-and-swap Technik eignet sich besonders gut für Klassen, die dynamische Ressourcen verwalten.

```
class Texture
{
public:
    Texture();
};
```

■ Welche Konstrukte sind für diese Klassendefinition vorhanden?

- | | |
|-----------------------------|------------------------------------|
| ■ Standardkonstruktor | ■ Verschiebungskonstruktor |
| ■ Kopierkonstruktor | ■ Verschiebungs-Zuweisungsoperator |
| ■ Kopier-Zuweisungsoperator | ■ Destruktor |